

claude-windows / 8d4866bf-8edd-4da9-909e-022ddee12675

Metadata

```
- Source: `claude-windows`  
- Kind: `claude`  
- Source file: `C:\Users\avidu\.claude\projects\C--Users-avidu-Projects-badminton-highlight-indexer\8d4866bf-8edd-4da9-909e-022ddee12675\subagents\workflows\wf_e27060ad-65f\agent-ad7d303ff1a75c770.jsonl`  
- SHA-256: `4f29cd0afdab22726b92092a69eccb127451f05cb93049e16bf8c53e8e3f1b58`  
- Source modified: `2026-06-19T16:44:38+00:00`  
- Imported at: `2026-07-05T16:48:25+00:00`  
- project: `wf_e27060ad-65f`  
- session_id: `8d4866bf-8edd-4da9-909e-022ddee12675`
```

Transcript

1. user (2026-06-19T16:40:08.114Z)

You are adversarially reviewing a technical claim in a proposed PR. Verify against REAL code; quote exact lines. Try BOTH to confirm and refute. This is the highest-stakes claim because it touches rally-END boundary logic.

PR CLAIM B ? "Sensitivity to Sparse Tracking in `_trim_inactive_tail` (tracknet_runner.py)":

- > The inactivity trimmer finds the rally end by looking for the last frame where the trailing `trail_frames` window has sufficient density of fast frames (`fast_sum / win_count >= min_density`).
- > Hidden issue: `win_count` is the number of `VISIBLE`, active frames in that boundary, not total elapsed time. If the detector loses the shuttle for 1.8s and picks up 1 fast frame at the end, `win_count` is ~1-2, so `fast_sum/win_count` could easily be 1.0 (100% density).
- > Impact: sparse tracking at clip end might prevent the tail trimmer from cutting dead-time, artificially inflating the candidate window END. Somewhat mitigated by `merge_gap` breaking trajectories, but remains a boundary quirk.

The PLAN proposes: "Update `_trim_inactive_tail` to factor the temporal length (or total expected frames in the trailing window) into the density calculation, rather than purely relying on `win_count` (number of visible frames)."

TASKS:

1. Read backend/pipeline/detectors/tracknet_runner.py: the `_trim_inactive_tail` def (~line 402), its caller (~line 381), and how groups are formed (the `merge_gap` grouping upstream). Quote exact code: how are `win_count`, `fast_sum`, `min_density`, `trail_frames`, `rally_thresh` actually computed? Is "density over visible frames" accurate?
2. Critically test the "win_count ~1-2 ? 100% density inflates the end" scenario. Walk the actual loop. Does `merge_gap` grouping mean a 1.8s gap would already SPLIT the group so the trailing isolated frame is in a DIFFERENT group (so it can't inflate THIS window's end)? Quote the merge logic and the relevant config default for `merge_gap`. Is the claimed scenario actually reachable in production, or is it pre-empted by grouping?
3. Read tests/test_tracknet_runner.py (test_trim_inactive_tail_*). What behavior is locked in?
4. Assess the proposed fix ("factor temporal length into density"): is it well-specified enough to implement safely? CRITICALLY: would changing the density denominator from visible-frame-count to temporal-length SHIFT rally-END boundaries on normal clips (not just the edge case)? The repo's stated #1 quality gap is rally-END detection, and the owner mandates that any change to default boundary behavior needs a measured Launch Report + dual-set regression, NOT a blind "harden the edge case" code change. Flag whether this fix is a silent boundary-behavior change masquerading as a bug fix.
5. Open question in the plan: "fall back to existing behavior for very few frames, or strictly enforce minimum temporal density?" ? assess whether the proposal has actually thought this through.

Return the structured verdict. `claim_id` = "B-trim-tail".

2. assistant (2026-06-19T16:40:10.550Z)

I'll review this claim against the real code. Let me start by reading the relevant files.

3. user (2026-06-19T16:40:12.266Z)

```
430de76 docs(rally): capture the shuttle-floor-contact rally-END signal (owner note)
e358b28 feat(eval): prominent-court window gate ? drop multicourt background-court rallies
(default-OFF) (#243)
9736a3b docs: re-ground stale doc claims to the current code (multi-agent audit) (#242)
---
docs/court-floor-rally-end
```

4. user (2026-06-19T16:40:12.863Z)

```
1      """TrackNet shuttle-trajectory runner (WSL2 subprocess adapter).
2
3      This bridges the Windows-side indexer to a TensorFlow TrackNetV4 install living
4      inside WSL2 (ADR-001 in docs/DECISIONS.md). The flow is:
5
6          1. Translate the Windows video path to its WSL-visible form (and optionally
7             stage it into the WSL filesystem to avoid slow /mnt/c per-frame reads).
8          2. Invoke ``src/predict.py`` inside the TrackNetV4 conda env via ``wsl``.
9          3. Read back the per-frame ``Frame,Visibility,X,Y`` CSV it writes.
10         4. Convert that trajectory into high-motion "action windows" ? the same
11            candidate-rally shape HybridYoloSegmenter produces ? so the rest of the
12            pipeline (AI handoff, DB population) is unchanged.
13
14      Nothing here imports TensorFlow; all model code stays in WSL.
15      """
16
17      import os
18      import csv
19      import shlex
20      import logging
21      import subprocess
22      from dataclasses import dataclass
23      from typing import List, Dict, Any, Optional, Tuple
24
25      from backend.pipeline.detectors.base import DetectorRunner, WslCommandMixin,
wsl_tilde_quote
26
27      logger = logging.getLogger(__name__)
28
29
30      @dataclass
31      class TrackNetConfig:
32          """Resolved settings for a TrackNet WSL run (built from config.json ->
indexing.tracknet)."""
33          distro: str = "Ubuntu"
34          repo_dir: str = "~/models/TrackNetV4"          # WSL path to the cloned repo
35          conda_sh: str = "~/miniconda3/etc/profile.d/conda.sh"
36          conda_env: str = "TrackNetV4"
37          weights_path: str = ""                        # WSL path to the .h5/.keras weights
(REQUIRED)
38          queue_length: int = 5
39          stage_in_wsl: bool = True                     # copy video into WSL fs first
(perf)
40          wsl_stage_dir: str = "~/clips"                # where staged videos/outputs live
in WSL
41          timeout_sec: int = 1800                      # 30 min; kills a hung call (0 = no
timeout)
42          keep_frames: bool = False                    # retain staged video after success
(debug only)
43          min_free_gb: float = 0.0                     # warn if WSL free space below this
(0 = off)
44
45      @classmethod
46      def from_indexing_cfg(cls, idx_cfg: Dict[str, Any]) -> "TrackNetConfig":
```

```

47         tn = (idx_cfg or {}).get("tracknet", {}) or {}
48         return cls(
49             distro=tn.get("wsl_distro", "Ubuntu"),
50             repo_dir=tn.get("repo_dir", "~/models/TrackNetV4"),
51             conda_sh=tn.get("conda_sh", "~/miniconda3/etc/profile.d/conda.sh"),
52             conda_env=tn.get("conda_env", "TrackNetV4"),
53             weights_path=tn.get("weights_path", ""),
54             queue_length=int(tn.get("queue_length", 5)),
55             stage_in_wsl=bool(tn.get("stage_in_wsl", True)),
56             wsl_stage_dir=tn.get("wsl_stage_dir", "~/clips"),
57             timeout_sec=int(tn.get("timeout_sec", 1800)),
58             keep_frames=bool(tn.get("keep_frames", False)),
59             min_free_gb=float(tn.get("min_free_gb", 0.0)),
60         )
61
62
63     def to_wsl_mnt_path(win_path: str) -> str:
64         """Translate a Windows path (C:\\Users\\x) to its WSL /mnt form (/mnt/c/Users/x).
65
66         Already-POSIX paths (starting with / or ~) are returned unchanged so the
67         same helper is safe to call on either kind of path.
68         """
69         p = str(win_path)
70         if p.startswith("/") or p.startswith("~"):
71             return p
72         p = p.replace("\\", "/")
73         if len(p) >= 2 and p[1] == ":":
74             drive = p[0].lower()
75             return f"/mnt/{drive}{p[2:]}"
76         return p
77
78
79     @dataclass
80     class TrajectoryPoint:
81         frame: int
82         visible: bool
83         x: float
84         y: float
85
86
87     class TrackNetRunner(WslCommandMixin, DetectorRunner):
88         """Runs TrackNet predict.py in WSL and turns the output CSV into action windows.
89
90         Implements the common DetectorRunner contract so a future Linux-native or
91         alternative trajectory runner can be substituted without touching the hybrids.
92         """
93
94         def __init__(self, cfg: TrackNetConfig):
95             self.cfg = cfg
96
97         def healthcheck(self) -> Tuple[bool, str]:
98             """Verify the WSL env is usable: conda env exists, TF imports, GPU visible.
99
100             Returns (ok, message). Cheap to call before a long run.
101             """
102             cmd = (
103                 f"conda activate {self.cfg.conda_env} && "
104                 f"python -c \"import tensorflow as tf; "
105                 f"print('TF', tf.__version__, 'GPUs', "
106                 f"len(tf.config.list_physical_devices('GPU')))\n"
107             )
108             try:
109                 res = self._wsl_bash(cmd)
110             except FileNotFoundError:
111                 return False, "wsl` not found ? is this running on Windows with WSL2

```

```

installed?"
111         except subprocess.TimeoutExpired:
112             return False, "WSL healthcheck timed out."
113         out = (res.stdout or "").strip()
114         if res.returncode != 0:
115             return False, f"WSL env check failed: {(res.stderr or out).strip()}"
116         return True, out
117
118     # ----- inference
119
120     def run_predict(self, video_win_path: str, output_win_dir: str,
121                    video_id: Optional[str] = None) -> Optional[str]:
122         """Run predict.py on a video. Returns the Windows path to the produced CSV, or
123         None.
124
125         ``output_win_dir`` is a Windows directory (under the repo's output/) that
126         WSL writes into via its /mnt view, so the Windows process can read the CSV
127         back with no copy. ``video_id`` (file MD5) namespaces the staged video ? and
128         therefore predict.py's ``<stem>_predict.csv`` output ? so two videos that share
129         a filename cannot collide on the output CSV.
130         """
131         if not self.cfg.weights_path:
132             logger.error(
133                 "TrackNet weights_path is not set. Set indexing.tracknet.weights_path "
134                 "in config.json to the WSL path of your trained TrackNetV4 weights."
135             )
136             return None
137
138         # Resolve relative dirs BEFORE the /mnt translation ? to_wsl_mnt_path passes
139         # relative paths through unchanged, so WSL would resolve them against the
140         # predict.py cwd instead of the repo (same bug class as wasb_runner).
141         output_win_dir = os.path.abspath(output_win_dir)
142         os.makedirs(output_win_dir, exist_ok=True)
143         # Normalize separators so basename/splitext work when tests (or collector)
144         # pass Windows-style paths on a Linux host.
145         video_win_path = str(video_win_path).replace("\\", "/")
146         base = os.path.basename(video_win_path)
147         stem, ext = os.path.splitext(base)
148
149         # predict.py only accepts .avi/.mp4 and names the CSV "<stem>_predict.csv".
150         if ext.lower() not in (".mp4", ".avi"):
151             logger.error("TrackNet predict.py only supports .mp4/.avi (got %s)", ext)
152             return None
153
154         wsl_out = to_wsl_mnt_path(output_win_dir)
155         # Namespace by video_id so two same-filename videos don't collide on the CSV.
156         # predict.py derives its output name from the (staged) video stem, so staging
157         # under the namespaced name propagates into "<key>_predict.csv".
158         key = f"{stem}_{video_id[:12]}" if video_id else stem
159         expected_csv_win = os.path.join(output_win_dir, f"{key}_predict.csv")
160
161         # Resolve the video path WSL will read.
162         if self.cfg.stage_in_wsl:
163             staged = self._stage_video(video_win_path, f"{key}{ext}")
164             if staged is None:
165                 return None
166             wsl_video = staged
167         else:
168             # No staging: predict.py reads /mnt directly and writes
169             # "<stem>_predict.csv".
170             # We cannot rename its output, so fall back to the un-namespaced name.
171             wsl_video = to_wsl_mnt_path(video_win_path)
172             expected_csv_win = os.path.join(output_win_dir, f"{stem}_predict.csv")
173
174         if self.cfg.min_free_gb > 0:

```

```

173         free = self._wsl_free_gb(self.cfg.wsl_stage_dir)
174         if free is not None and free < self.cfg.min_free_gb:
175             logger.warning("Low WSL disk: %.1f GB free at %s (< min_free_gb=%.1f); "
176                             "consider running tools/cleanup_caches.py.",
177                             free, self.cfg.wsl_stage_dir, self.cfg.min_free_gb)
178
179         cmd = (
180             f"conda activate {self.cfg.conda_env} && "
181             f"cd {self.cfg.repo_dir}/src && "
182             f"python predict.py "
183             f"--video_path {wsl_tilde_quote(wsl_video)} "
184             f"--model_weights {wsl_tilde_quote(self.cfg.weights_path)} "
185             f"--output_dir {wsl_tilde_quote(wsl_out)} "
186             f"--queue_length {self.cfg.queue_length}")
187         )
188         logger.info("Running TrackNet inference on %s ...", base)
189         try:
190             res = self._wsl_bash(cmd)
191         except subprocess.TimeoutExpired:
192             logger.error("TrackNet inference timed out after %ss.",
193                         self.cfg.timeout_sec)
194             return None
195
196         if res.returncode != 0:
197             logger.error("TrackNet predict.py failed:\n%s", (res.stderr or
198                 res.stdout)[-2000:])
199             return None
200
201         if not os.path.exists(expected_csv_win):
202             logger.error("TrackNet finished but expected CSV not found at %s",
203                 expected_csv_win)
204             return None
205         logger.info("TrackNet trajectory CSV: %s", expected_csv_win)
206
207         # Success ? the CSV is the durable output; drop the staged video copy (a pure,
208         # regenerable intermediate). On earlier failure we return above without
209         cleaning.
210         if self.cfg.stage_in_wsl and not self.cfg.keep_frames:
211             logger.info("Cache hygiene: removing staged video for %s "
212                         "(set indexing.tracknet.keep_frames=true to retain).", key)
213             self._wsl_rm_rf([wsl_video])
214             return expected_csv_win
215
216     def _stage_video(self, video_win_path: str, base: str) -> Optional[str]:
217         """Copy the input video into the WSL filesystem (avoids slow /mnt/c reads).
218
219         Returns the WSL path to the staged copy, or None on failure.
220         """
221         src_mnt = to_wsl_mnt_path(video_win_path)
222         reason = self.wsl_source_unreadable_reason(src_mnt)
223         if reason:
224             logger.error("Cannot stage video into WSL: %s", reason)
225             return None
226         dest = f"{self.cfg.wsl_stage_dir}/{base}"
227         cmd = f"mkdir -p {self.cfg.wsl_stage_dir} && cp {shlex.quote(src_mnt)} "
228         {wsl_tilde_quote(dest)} && echo OK"
229         try:
230             res = self._wsl_bash(cmd)
231         except subprocess.TimeoutExpired:
232             logger.error("Staging video into WSL timed out.")
233             return None
234         if res.returncode != 0 or "OK" not in (res.stdout or ""):
235             logger.error("Failed to stage video into WSL: %s", (res.stderr or
236                 res.stdout).strip())
237             return None

```

```

232         return dest
233
234     # ----- CSV -> windows
235
236     @staticmethod
237     def parse_trajectory_csv(csv_win_path: str) -> List[TrajectoryPoint]:
238         """Parse a TrackNet predict CSV (columns: Frame,Visibility,X,Y)."""
239         points: List[TrajectoryPoint] = []
240         with open(csv_win_path, newline="") as f:
241             reader = csv.DictReader(f)
242             for row in reader:
243                 try:
244                     points.append(TrajectoryPoint(
245                         frame=int(row["Frame"]),
246                         visible=int(row["Visibility"]) == 1,
247                         x=float(row["X"]),
248                         y=float(row["Y"]),
249                     ))
250                 except (KeyError, ValueError):
251                     continue
252         points.sort(key=lambda p: p.frame)
253         return points
254
255     @staticmethod
256     def _active_frames(points: List[TrajectoryPoint], fps: float, frame_width: float,
257                        velocity_thresh: float) -> List[Tuple[int, float]]:
258         """Per-frame ``(frame_idx, velocity)`` for frames where the shuttle is in
flight.
259
260         Computes each visible point's normalised velocity from the last *visible* point;
261         an absent shuttle breaks the trajectory. A frame qualifies when its velocity
exceeds
262         ``velocity_thresh``. ``fps``/``frame_width`` are expected to be already
defaulted by
263         the caller so identical floats feed the velocity math."""
264         active = [] # (frame_idx, velocity)
265         prev: Optional[TrajectoryPoint] = None
266         for p in points:
267             if not p.visible:
268                 prev = None # break the trajectory; absent shuttle = not in flight
269                 continue
270             if prev is not None:
271                 dframes = p.frame - prev.frame
272                 if dframes > 0:
273                     dist = ((p.x - prev.x) ** 2 + (p.y - prev.y) ** 2) ** 0.5
274                     velocity = (dist / frame_width) * (fps / dframes)
275                     if velocity > velocity_thresh:
276                         active.append((p.frame, velocity))
277             prev = p
278         return active
279
280     @staticmethod
281     def trajectory_to_action_windows(
282         points: List[TrajectoryPoint],
283         fps: float,
284         frame_width: float,
285         chunk_start: float = 0.0,
286         velocity_thresh: float = 0.02,
287         merge_gap: float = 2.0,
288         min_window_duration: float = 2.0,
289         chunk_index: Optional[int] = None,
290         max_window_duration: float = 0.0,
291         min_split_gap: float = 0.5,
292         window_hard_cap: bool = False,
293         window_inactivity_end: float = 0.0,

```

```

294         inactivity_min_density: float = 0.34,
295         inactivity_rally_thresh: float = 0.08,
296         window_blob_fallback: bool = False,
297         window_blob_factor: float = 4.0,
298     ) -> List[Dict[str, Any]]:
299         """Convert a shuttle trajectory into candidate rally windows.
300
301         A frame is "active" when the shuttle is visible AND its inter-frame
302         displacement (normalised by frame width, per second) exceeds
303         ``velocity_thresh`` ? i.e. the shuttle is in flight. Active frames within
304         ``merge_gap`` seconds of each other are grouped into one window; short
305         windows are dropped. Mirrors HybridYoloSegmenter's windowing so the dict
306         shape feeding the AI-handoff phase is identical.
307
308         ``max_window_duration`` (0 = OFF, the default ? behaviour unchanged): a guard
against
309         the *over-merge* failure where "shuttle moving" is conflated with "rally in
progress"
310         and a single window swallows several rallies + the dead time between them. When
311         window longer than this is recursively SPLIT at its largest internal inactivity
gap (the
312         longest stretch with no in-flight shuttle ? the most likely rally boundary),
provided
313         that gap is at least ``min_split_gap`` seconds. This is the bounded-windowing
fix from
314         ``docs/RALLY_QUALITY_RESEARCH.md`` §6 (W1); it never merges, only cuts, so
recall cannot
315         drop, and it is config-gated default-OFF until measured on the golden set.
316
317         ``window_hard_cap`` (default OFF) makes ``max_window_duration`` a TRUE cap: when
an
318         over-long window has NO internal inactivity gap to split on (a continuously-
active
319         trajectory ? the shuttle never stops moving, e.g. the real-footage
``mahadevpura-1``
320         blob where the gap-splitter alone left a single 174 s window), it is recursively
321         hard-chopped at its temporal midpoint until every piece is ? the cap. Still only
cuts
322         (never merges), so recall cannot drop; gated default-OFF until measured.
323
324         ``window_blob_fallback`` (default OFF ? the R5 last-resort blob splitter):
unlike
325         ``window_hard_cap`` (which midpoint-chops BEFORE the R4 trim, so it fires on
every gap-less
326         over-cap window and over-segments normal clips), this runs AFTER the R4
inactivity trim and
327         force-chops ONLY a genuine *blob* ? a group still longer than
``window_blob_factor`` x
328         ``max_window_duration`` once the gap-split and the principled rally-end trim
have both had
329         their chance (e.g. mahadevpura-1's ~65 s residual ? 11x a 6 s cap; a 7 s rally
is left
330         alone). The blob is midpoint-chopped down to ? the cap, those forced cuts are
logged, and
331         each resulting window is flagged ``forced_cut=True`` ? surfacing the clip as one
that needs
332         rally-STATE cues (net-crossings / two-sided motion), not a bigger cap. Only cuts
(recall
333         can't drop). Requires ``max_window_duration`` > 0; gated default-OFF until
measured.
334         """
335         if fps <= 0:
336             fps = 30.0
337         if frame_width <= 0:

```

```

338         frame_width = 1280.0
339
340         active = TrackNetRunner._active_frames(points, fps, frame_width,
velocity_thresh)
341
342         if not active:
343             return []
344
345         max_gap_frames = int(merge_gap * fps)
346
347         def _finalize(group: List[Tuple[int, float]]) -> Dict[str, Any]:
348             vels = [v for _, v in group]
349             return {
350                 "start": group[0][0] / fps + chunk_start,
351                 "end": group[-1][0] / fps + chunk_start,
352                 "active_frame_count": len(group),
353                 "peak_velocity": max(vels),
354                 "mean_velocity": sum(vels) / len(vels),
355                 "track_id_count": 1, # single shuttle; kept for window-shape parity
356                 "chunk_index": chunk_index,
357             }
358
359         groups: List[List[Tuple[int, float]]] = []
360         group = [active[0]]
361         for af in active[1:]:
362             if af[0] - group[-1][0] <= max_gap_frames:
363                 group.append(af)
364             else:
365                 groups.append(group)
366                 group = [af]
367         groups.append(group)
368
369         if max_window_duration > 0:
370             groups = TrackNetRunner._split_overlong_groups(
371                 groups, int(max_window_duration * fps), int(min_split_gap * fps),
372                 hard_cap=window_hard_cap)
373
374         # R4 ? rally-END inactivity trim (default OFF). Applied AFTER the cap split so
each rally
375         # piece has its own post-rally drift tail removed: the velocity windowing keeps
a window
376         # "active" while the shuttle drifts/gets-picked-up above threshold, over-
extending the END
377         # (the measured gap vs golden). Trim back to the last frame whose trailing
window is still
378         # dense with motion (rally on); a sustained low-density run = rally over. Only
cuts.
379         if window_inactivity_end > 0:
380             tw = int(window_inactivity_end * fps)
381             groups = [TrackNetRunner._trim_inactive_tail(g, tw, inactivity_min_density,
382                                                         inactivity_rally_thresh) for g
in groups]
383
384         # R5 ? blob-fallback (default OFF). LAST resort: after the gap-split AND the R4
trim, any
385         # group still longer than the cap is a gap-less, rally-end-signal-less blob;
force-chop it to
386         # ? the cap at the midpoint and flag every piece, so the degenerate single-
window outcome is
387         # avoided and the clip is surfaced (logged) as needing rally-STATE cues, not a
bigger cap.
388         forced_flags = [False] * len(groups)
389         if window_blob_fallback and max_window_duration > 0:
390             groups, forced_flags = TrackNetRunner._blob_fallback_chop(
391                 groups, int(max_window_duration * fps), window_blob_factor)

```



```

392
393     windows = []
394     for g, forced in zip(groups, forced_flags):
395         w = _finalize(g)
396         if forced:
397             w["forced_cut"] = True
398         windows.append(w)
399     return [w for w in windows if (w["end"] - w["start"]) >= min_window_duration]
400
401     @staticmethod
402     def _trim_inactive_tail(group: List[Tuple[int, float]], trail_frames: int,
403                             min_density: float, rally_thresh: float) -> List[Tuple[int,
404 float]]:
405         """R4: trim the post-rally drift tail. After a rally the shuttle keeps moving
406         *slowly*
407         (picked up / walked / knock-up) above ``velocity_thresh``, so the velocity
408         windowing
409         over-extends the END. A frame is "fast" (real rally motion) iff its velocity >
410         ``rally_thresh``
411         (higher than the active threshold). The rally is "on" at frame ``i`` while its
412         trailing
413         ``trail_frames`` window holds >= ``min_density`` fraction of fast frames; trim
414         back to the
415         LAST such frame ? everything after is sustained slow drift (rally over). Only
416         cuts (recall
417         can't rise); a window whose tail never goes slow is untouched, and if no fast-
418         dense point
419         exists at all the group is kept whole (fail-safe, no over-trim). O(n) two-
420         pointer; the START
421         is left alone (already ~Gemini-accurate per the n=2 boundary-error finding)."""
422         if trail_frames <= 0 or len(group) < 2:
423             return group
424         frames = [f for f, _ in group]
425         fast = [1 if v > rally_thresh else 0 for _, v in group]
426         lo = 0
427         fast_sum = 0
428         last_dense = -1
429         for i in range(len(frames)):
430             fast_sum += fast[i]
431             while frames[lo] <= frames[i] - trail_frames:
432                 fast_sum -= fast[lo]
433                 lo += 1
434             win_count = i - lo + 1
435             if win_count > 0 and fast_sum / win_count >= min_density:
436                 last_dense = i
437         return group[: last_dense + 1] if last_dense >= 0 else group
438
439     @staticmethod
440     def _midpoint_split_index(group: List[Tuple[int, float]]) -> int:
441         """Index of the active frame nearest the group's temporal midpoint (used to chop
442         a
443         gap-less group into two). Shared by the W1 hard-cap fallback and the R5 blob-
444         fallback."""
445         mid = (group[0][0] + group[-1][0]) / 2.0
446         split_i = min(range(1, len(group)), key=lambda i: abs(group[i][0] - mid))
447         return split_i
448
449     @staticmethod
450     def _split_overlong_groups(groups: List[List[Tuple[int, float]]], max_win_frames:
451 int,
452                             min_split_frames: int,
453                             hard_cap: bool = False) -> List[List[Tuple[int, float]]]:
454         """Recursively split any active-frame group longer than ``max_win_frames`` at
455         its largest
456         internal gap (? ``min_split_frames``) ? an inactivity-aware cut at the most

```

```

likely rally
444         boundary. Splitting only subdivides existing groups (never merges/extends), so
it cannot
445         invent or drop coverage; a group with no gap ? the floor is left intact (a
genuinely long
446         rally) UNLESS ``hard_cap`` is set, in which case such a group is hard-chopped at
its
447         temporal midpoint until every piece is ? ``max_win_frames`` (makes the cap a
true upper
448         bound on continuously-active trajectories). Iterative worklist to bound stack
depth.
449         if max_win_frames <= 0:
450             return groups
451         out: List[List[Tuple[int, float]]] = []
452         work = list(groups)
453         while work:
454             g = work.pop()
455             if len(g) < 2 or (g[-1][0] - g[0][0]) <= max_win_frames:
456                 out.append(g)
457                 continue
458             best_i, best_gap = -1, -1
459             for i in range(1, len(g)):
460                 gap = g[i][0] - g[i - 1][0]
461                 if gap > best_gap:
462                     best_gap, best_i = gap, i
463             if best_i < 0 or best_gap < min_split_frames:
464                 # Too long but no real dead-time gap. Default: keep as one (a genuine
long rally).
465                 # hard_cap: chop at the temporal midpoint so the cap is actually
enforced.
466                 if hard_cap:
467                     split_i = TrackNetRunner._midpoint_split_index(g)
468                     work.append(g[:split_i])
469                     work.append(g[split_i:])
470                 else:
471                     out.append(g)
472                     continue
473                 work.append(g[:best_i])
474                 work.append(g[best_i:])
475         out.sort(key=lambda grp: grp[0][0])
476         return out
477
478     @staticmethod
479     def _blob_fallback_chop(groups: List[List[Tuple[int, float]]], max_win_frames: int,
480                             blob_factor: float = 4.0
481                             ) -> Tuple[List[List[Tuple[int, float]]], List[bool]]:
482         """R5 blob splitter ? the gap-less, R4-survived residual. After the W1 gap-split
AND the R4
483         inactivity trim have each had their chance, a group still longer than
``blob_factor`` x
484         ``max_win_frames`` is a genuine continuously-active blob (no internal gap, no
rally-end
485         signal the principled rules could find ? the real-footage ``mahadevpura-1`` ~65
s residual ?
486         11x a 6 s cap). The factor is what keeps this SURGICAL: a normal long rally
(~1?2x the cap)
487         is left ALONE; only the degenerate blob qualifies. As a LAST RESORT the blob is
midpoint-chopped to ? the cap and every resulting piece is FLAGGED as a forced
cut, so the
489         degenerate single-window outcome is avoided and the clip is surfaced (logged) as
needing
490         rally-STATE cues ? net-crossings / two-sided motion ? not a bigger cap. Only
cuts (recall
491         can't drop). Returns ``(groups, forced_flags)`` aligned 1:1; ``forced_flags[i]``
is True iff

```

```

492         that piece came from a forced chop. Iterative worklist to bound stack depth."""
493     if max_win_frames <= 0:
494         return groups, [False] * len(groups)
495     blob_frames = max_win_frames * max(1.0, blob_factor)
496     out: List[List[Tuple[int, float]]] = []
497     flags: List[bool] = []
498     forced = 0
499     for g in groups:
500         if len(g) < 2 or (g[-1][0] - g[0][0]) <= blob_frames:
501             out.append(g)
502             flags.append(False)
503             continue
504         pieces: List[List[Tuple[int, float]]] = []
505         work = [g]
506         while work:
507             h = work.pop()
508             if len(h) < 2 or (h[-1][0] - h[0][0]) <= max_win_frames:
509                 pieces.append(h)
510                 continue
511             split_i = TrackNetRunner._midpoint_split_index(h)
512             work.append(h[:split_i])
513             work.append(h[split_i:])
514         pieces.sort(key=lambda grp: grp[0][0])
515         out.extend(pieces)
516         flags.extend([True] * len(pieces))
517         forced += len(pieces) - 1
518     if forced:
519         logger.warning(
520             "R5 blob-fallback: forced %d midpoint cut(s) on continuously-active
group(s) with "
521             "no rally-end signal ? this clip needs rally-STATE cues (net-crossings /
two-sided "
522             "motion), not a bigger cap", forced)
523     return out, flags
524

```

5. assistant (2026-06-19T16:40:15.275Z)

Now let me read the config defaults and the tests. Let me find where these parameters are configured and the test file.

6. user (2026-06-19T16:40:16.253Z)

```

1     """Tests for the TrackNet runner's pure-Python pieces (path translation, CSV
2     parsing, trajectory -> action-window conversion). The WSL subprocess invocation
3     itself is not exercised here ? these run anywhere, no WSL/TensorFlow needed."""
4
5     import os
6     import subprocess
7     import textwrap
8     from unittest.mock import patch
9
10    from backend.pipeline.detectors.tracknet_runner import (
11        TrackNetRunner,
12        TrackNetConfig,
13        TrajectoryPoint,
14        to_wsl_mnt_path,
15    )
16    from backend.pipeline.detectors.base import DetectorRunner
17    from backend.pipeline.detectors.wasb_runner import WasbRunner, WasbConfig
18
19
20    def _proc(returncode=0, stdout="", stderr=""):
21        return subprocess.CompletedProcess(args=[], returncode=returncode,
22            stdout=stdout, stderr=stderr)
23

```

```

23
24
25 def test_to_wsl_mnt_path_windows():
26     assert to_wsl_mnt_path(r"C:\Users\avidu\v.mp4") == "/mnt/c/Users/avidu/v.mp4"
27     assert to_wsl_mnt_path(r"D:\clips\m.mp4") == "/mnt/d/clips/m.mp4"
28
29
30 def test_to_wsl_mnt_path_passthrough():
31     # Already-POSIX paths are left alone.
32     assert to_wsl_mnt_path("/home/avidullu/clips/v.mp4") == "/home/avidullu/clips/v.mp4"
33     assert to_wsl_mnt_path("~/clips/v.mp4") == "~/clips/v.mp4"
34
35
36 def test_config_from_indexing_cfg():
37     cfg = TrackNetConfig.from_indexing_cfg({"tracknet": {
38         "wsl_distro": "Ubuntu-22.04", "conda_env": "TN", "weights_path": "/w/m.h5",
39         "queue_length": 7, "stage_in_wsl": False,
40     }})
41     assert cfg.distro == "Ubuntu-22.04"
42     assert cfg.conda_env == "TN"
43     assert cfg.weights_path == "/w/m.h5"
44     assert cfg.queue_length == 7
45     assert cfg.stage_in_wsl is False
46
47
48 def test_config_defaults_when_absent():
49     cfg = TrackNetConfig.from_indexing_cfg({})
50     assert cfg.distro == "Ubuntu"
51     assert cfg.conda_env == "TrackNetV4"
52     assert cfg.weights_path == "" # must be supplied by the user
53
54
55 def test_parse_trajectory_csv(tmp_path):
56     csv_file = tmp_path / "v_predict.csv"
57     csv_file.write_text(textwrap.dedent("""\
58         Frame,Visibility,X,Y
59         0,0,0,0
60         1,1,100,200
61         2,1,140,210
62         3,0,0,0
63     """))
64     pts = TrackNetRunner.parse_trajectory_csv(str(csv_file))
65     assert len(pts) == 4
66     assert pts[0].visible is False
67     assert pts[1].visible is True and pts[1].x == 100 and pts[1].y == 200
68     assert pts[3].visible is False
69
70
71 def test_trajectory_to_action_windows_detects_moving_shuttle():
72     # 30 fps, 1280px wide. Shuttle moves ~60px/frame while visible (frames 0..29) =>
73     # well above the velocity threshold; then disappears. Expect one window ~1s long.
74     fps, width = 30.0, 1280.0
75     pts = [TrajectoryPoint(frame=i, visible=True, x=100 + 60 * i, y=300) for i in
76 range(30)]
77     pts += [TrajectoryPoint(frame=i, visible=False, x=0, y=0) for i in range(30, 60)]
78
79     windows = TrackNetRunner.trajectory_to_action_windows(
80         pts, fps=fps, frame_width=width, velocity_thresh=0.02,
81         merge_gap=2.0, min_window_duration=0.5, chunk_index=0,
82     )
83     assert len(windows) == 1
84     w = windows[0]
85     assert w["start"] < w["end"]
86     assert w["active_frame_count"] > 0
87     assert w["peak_velocity"] > 0.02

```

```

87         assert w["chunk_index"] == 0
88
89
90     def test_trajectory_to_action_windows_ignores_stationary_shuttle():
91         # Visible but barely moving (1px/frame) => below threshold => no windows.
92         pts = [TrajectoryPoint(frame=i, visible=True, x=500 + i, y=300) for i in range(60)]
93         windows = TrackNetRunner.trajectory_to_action_windows(
94             pts, fps=30.0, frame_width=1280.0, velocity_thresh=0.05,
95             merge_gap=2.0, min_window_duration=0.5,
96         )
97         assert windows == []
98
99
100    def test_trajectory_to_action_windows_splits_on_gap():
101        # Two bursts of motion separated by a long invisible gap => two windows.
102        burst1 = [TrajectoryPoint(frame=i, visible=True, x=100 + 60 * i, y=300) for i in
103            range(20)]
104        gap = [TrajectoryPoint(frame=i, visible=False, x=0, y=0) for i in range(20, 140)]
105        burst2 = [TrajectoryPoint(frame=i, visible=True, x=100 + 60 * (i - 140), y=300)
106            for i in range(140, 160)]
107        windows = TrackNetRunner.trajectory_to_action_windows(
108            burst1 + gap + burst2, fps=30.0, frame_width=1280.0,
109            velocity_thresh=0.02, merge_gap=2.0, min_window_duration=0.3,
110        )
111        assert len(windows) == 2
112        assert windows[0]["end"] < windows[1]["start"]
113
114    # ---- W1: bounded windowing (max_window_duration split)
115    ----- #
116    def _moving(a, b): # visible, fast-moving shuttle over frames [a, b)
117        return [TrajectoryPoint(frame=i, visible=True, x=100 + 60 * (i - a), y=300) for i in
118            range(a, b)]
119
120    def _invisible(a, b):
121        return [TrajectoryPoint(frame=i, visible=False, x=0, y=0) for i in range(a, b)]
122
123    # Two rallies merged across a sub-merge_gap dead time ? ONE over-long window without the
124    cap.
125    _MERGED PTS = _moving(0, 121) + _invisible(121, 162) + _moving(162, 243) # ~8s, 1.4s
126    internal gap
127
128    def test_max_window_duration_off_keeps_over_merge():
129        # Default (max_window_duration=0): the two rallies stay merged into one window.
130        w = TrackNetRunner.trajectory_to_action_windows(
131            _MERGED PTS, fps=30.0, frame_width=1280.0, velocity_thresh=0.02,
132            merge_gap=2.0, min_window_duration=0.3)
133        assert len(w) == 1 # over-merge reproduced (the failure the cap fixes)
134        assert (w[0]["end"] - w[0]["start"]) > 7.0
135
136    def test_max_window_duration_splits_at_inactivity_gap():
137        # With a 4s cap the 8s merged window is split at its 1.4s internal dead-time gap ? 2
138        windows.
139        w = TrackNetRunner.trajectory_to_action_windows(
140            _MERGED PTS, fps=30.0, frame_width=1280.0, velocity_thresh=0.02,
141            merge_gap=2.0, min_window_duration=0.3, max_window_duration=4.0,
142            min_split_gap=0.5)
143        assert len(w) == 2
144        assert w[0]["end"] < w[1]["start"]
145        assert all((x["end"] - x["start"]) <= 5.0 for x in w)

```

```

145
146     def test_max_window_duration_preserves_genuine_long_rally():
147         # A single continuous 10s rally (no internal gap ? min_split_gap) is NOT split, even
148         # though it exceeds the cap ? bounded windowing only cuts at real dead time.
149         w = TrackNetRunner.trajectory_to_action_windows(
150             _moving(0, 301), fps=30.0, frame_width=1280.0, velocity_thresh=0.02,
151             merge_gap=2.0, min_window_duration=0.3, max_window_duration=4.0,
min_split_gap=0.5)
152         assert len(w) == 1
153         assert (w[0]["end"] - w[0]["start"]) > 9.0
154
155
156     def test_split_overlong_groups_pure():
157         # Direct check of the splitter: a group with one big internal gap splits there.
158         grp = [(i, 1.0) for i in range(0, 20)] + [(i, 1.0) for i in range(60, 80)] # gap
20..60
159         out = TrackNetRunner._split_overlong_groups([grp], max_win_frames=30,
min_split_frames=10)
160         assert len(out) == 2 and out[0][-1][0] == 19 and out[1][0][0] == 60
161         # No qualifying gap (all consecutive) ? unchanged even when over length.
162         solid = [(i, 1.0) for i in range(0, 100)]
163         assert TrackNetRunner._split_overlong_groups([solid], 30, 10) == [solid]
164
165
166     # ---- W1 hard-cap fallback: enforce the cap on continuously-active trajectories
----- #
167     def test_window_hard_cap_chops_continuous_rally():
168         # The same continuous ~10s rally that bounded-windowing alone LEAVES intact (no
internal
169         # gap) is, with window_hard_cap=True, chopped at the midpoint until every piece ?
the cap.
170         w = TrackNetRunner.trajectory_to_action_windows(
171             _moving(0, 301), fps=30.0, frame_width=1280.0, velocity_thresh=0.02,
172             merge_gap=2.0, min_window_duration=0.3, max_window_duration=4.0,
min_split_gap=0.5,
173             window_hard_cap=True)
174         assert len(w) >= 3 # the 10s blob is broken
up
175         assert all((x["end"] - x["start"]) <= 4.01 for x in w) # every piece within the
cap
176         assert w[0]["start"] <= 0.1 and w[-1]["end"] >= 9.0 # coverage preserved
(only cuts)
177
178
179     def test_split_overlong_groups_hard_cap_pure():
180         solid = [(i, 1.0) for i in range(0, 100)] # 99-frame span, no
internal gap
181         # default: kept whole; hard_cap: chopped into pieces each spanning ? max_win_frames
182         assert TrackNetRunner._split_overlong_groups([solid], 30, 10) == [solid]
183         out = TrackNetRunner._split_overlong_groups([solid], 30, 10, hard_cap=True)
184         assert len(out) >= 4
185         assert all((g[-1][0] - g[0][0]) <= 30 for g in out)
186         # only cuts, never drops/duplicates: the pieces partition the original frames
exactly
187         assert sorted(f for g in out for f, _ in g) == list(range(0, 100))
188
189
190     # ---- R4: rally-END inactivity trim (drop the post-rally slow-drift tail)
----- #
191     def test_trim_inactive_tail_drops_slow_drift():
192         # fast rally (vel 0.5) frames 0..60, then slow drift (vel 0.03, still > active 0.02)
61..200.
193         group = [(i, 0.5) for i in range(0, 61)] + [(i, 0.03) for i in range(61, 201)]
194         out = TrackNetRunner._trim_inactive_tail(group, trail_frames=30, min_density=0.3,
rally_thresh=0.2)

```

```

195         assert out[0][0] == 0                                # start untouched
196         assert out[-1][0] < 100                              # the pure-drift tail (frames
100..200) is trimmed off
197         assert len(group) - len(out) >= 100                  # ~all the slow drift cut (kept only a
short decay tail)
198
199
200     def test_trim_inactive_tail_keeps_continuous_rally_and_is_noop_when_off():
201         allfast = [(i, 0.5) for i in range(0, 100)]
202         assert TrackNetRunner._trim_inactive_tail(allfast, 30, 0.3, 0.2) == allfast    #
fully-fast: untouched
203         assert TrackNetRunner._trim_inactive_tail(allfast, 0, 0.3, 0.2) == allfast    #
trail=0: OFF/no-op
204
205
206     def test_window_inactivity_end_trims_through_api():
207         # one window of fast motion then slow drift (both visible & > velocity_thresh ? one
group).
208         fast = [TrajectoryPoint(frame=i, visible=True, x=100 + 40 * (i % 5), y=300) for i in
range(0, 90)]
209         slow = [TrajectoryPoint(frame=i, visible=True, x=100 + (i % 2), y=300) for i in
range(90, 300)]
210         pts = fast + slow
211         off = TrackNetRunner.trajectory_to_action_windows(
212             pts, fps=30.0, frame_width=1280.0, velocity_thresh=0.02, merge_gap=2.0,
min_window_duration=0.3)
213         on = TrackNetRunner.trajectory_to_action_windows(
214             pts, fps=30.0, frame_width=1280.0, velocity_thresh=0.02, merge_gap=2.0,
min_window_duration=0.3,
215             window_inactivity_end=1.0, inactivity_rally_thresh=0.2,
inactivity_min_density=0.3)
216         assert on and off
217         assert on[0]["end"] < off[0]["end"]                    # R4 trims the drift tail ? earlier END
218         assert on[0]["start"] == off[0]["start"]              # START unchanged
219
220
221     # ---- R5: blob-fallback (force-split the continuously-active blob, default OFF)
----- #
222     def test_blob_fallback_chop_pure():
223         blob = [(i, 1.0) for i in range(0, 200)]              # 199-frame span, no gap = 6.6× a
30-frame cap
224         out, flags = TrackNetRunner._blob_fallback_chop([blob], max_win_frames=30,
blob_factor=4.0)
225         assert len(out) >= 4 and all(flags)                    # force-chopped, every
piece flagged
226         assert all((g[-1][0] - g[0][0]) <= 30 for g in out)    # every piece within
the cap
227         assert sorted(f for g in out for f, _ in g) == list(range(0, 200)) # partition
(only cuts)
228
229
230     def test_blob_fallback_factor_leaves_normal_long_rally_alone():
231         # 79-frame span = 2.6× a 30-frame cap ? BELOW the 4× blob threshold ? left ALONE
(the gate that
232         # keeps R5 surgical; this is the over-cap-but-not-a-blob case hard_cap would wrongly
chop).
233         normal = [(i, 1.0) for i in range(0, 80)]
234         out, flags = TrackNetRunner._blob_fallback_chop([normal], max_win_frames=30,
blob_factor=4.0)
235         assert out == [normal] and flags == [False]
236
237
238     def test_window_blob_fallback_rescues_blob_through_api():
239         # A continuous ~10s blob (no gap) the cap alone leaves whole: blob_fallback breaks
it up + flags.

```

```

240         pts = _moving(0, 301)                                     # 10s continuous, no
internal gap
241         off = TrackNetRunner.trajectory_to_action_windows(
242             pts, fps=30.0, frame_width=1280.0, velocity_thresh=0.02, merge_gap=2.0,
243             min_window_duration=0.3, max_window_duration=2.0)      # cap 2s, blob_fallback
OFF
244         on = TrackNetRunner.trajectory_to_action_windows(
245             pts, fps=30.0, frame_width=1280.0, velocity_thresh=0.02, merge_gap=2.0,
246             min_window_duration=0.3, max_window_duration=2.0,
247             window_blob_fallback=True, window_blob_factor=4.0)    # 10s > 4x2s threshold ?
a blob
248         assert len(off) == 1 and (off[0]["end"] - off[0]["start"]) > 9.0 # cap alone
leaves it whole
249         assert len(on) >= 4                                           # the blob is broken up
250         assert all((w["end"] - w["start"]) <= 2.01 for w in on)      # every piece within the
cap
251         assert all(w.get("forced_cut") for w in on)                  # every piece flagged a
forced cut
252
253
254     def test_window_blob_fallback_noop_without_cap():
255         # blob_fallback requires a cap to define "over-cap"; with max_window_duration=0 it's
a no-op.
256         w = TrackNetRunner.trajectory_to_action_windows(
257             _moving(0, 301), fps=30.0, frame_width=1280.0, velocity_thresh=0.02,
merge_gap=2.0,
258             min_window_duration=0.3, window_blob_fallback=True)
259         assert len(w) == 1 and not w[0].get("forced_cut")
260
261
262     # ----- WSL subprocess glue
263
264     def _runner():
265         return TrackNetRunner(TrackNetConfig(weights_path="/w/model.h5"))
266
267
268     def test_healthcheck_ok():
269         r = _runner()
270         with patch.object(r, "_wsl_bash", return_value=_proc(0, stdout="TF 2.17.0 GPUs 1")):
271             ok, msg = r.healthcheck()
272             assert ok is True
273             assert "TF 2.17.0" in msg
274
275
276     def test_healthcheck_failure_returns_message():
277         r = _runner()
278         with patch.object(r, "_wsl_bash", return_value=_proc(1,
stderr="ModuleNotFoundError")):
279             ok, msg = r.healthcheck()
280             assert ok is False
281             assert "ModuleNotFoundError" in msg
282
283
284     def test_healthcheck_no_wsl_binary():
285         r = _runner()
286         with patch.object(r, "_wsl_bash", side_effect=FileNotFoundError()):
287             ok, msg = r.healthcheck()
288             assert ok is False
289             assert "wsl" in msg.lower()
290
291
292     def test_run_predict_requires_weights():
293         r = TrackNetRunner(TrackNetConfig(weights_path="")) # no weights configured
294         assert r.run_predict(r"C:\v\clip.mp4", "out") is None
295

```



```

296
297 def test_run_predict_rejects_bad_extension(tmp_path):
298     r = _runner()
299     assert r.run_predict(r"C:\v\clip.mkv", str(tmp_path)) is None
300
301
302 def test_run_predict_success(tmp_path):
303     r = TrackNetRunner(TrackNetConfig(weights_path="/w/model.h5", stage_in_wsl=False))
304     out_dir = str(tmp_path)
305     # predict.py would write "<stem>_predict.csv"; create it so the existence check
306     csv_path = os.path.join(out_dir, "clip_predict.csv")
307     with patch.object(r, "_wsl_bash", return_value=_proc(0, stdout="Prediction
complete.")):
308         open(csv_path, "w").close()
309         result = r.run_predict(r"C:\v\clip.mp4", out_dir)
310         assert result == csv_path
311
312
313 def test_run_predict_failure_returns_none(tmp_path):
314     r = TrackNetRunner(TrackNetConfig(weights_path="/w/model.h5", stage_in_wsl=False))
315     with patch.object(r, "_wsl_bash", return_value=_proc(1, stderr="CUDA error")):
316         assert r.run_predict(r"C:\v\clip.mp4", str(tmp_path)) is None
317
318
319 def test_run_predict_missing_output_csv(tmp_path):
320     """Exit 0 but no CSV produced should be treated as failure."""
321     r = TrackNetRunner(TrackNetConfig(weights_path="/w/model.h5", stage_in_wsl=False))
322     with patch.object(r, "_wsl_bash", return_value=_proc(0, stdout="done")):
323         assert r.run_predict(r"C:\v\clip.mp4", str(tmp_path)) is None
324
325
326 def test_stage_video_success():
327     r = _runner()
328     # Two WSL calls now: the readability preflight (echo READABLE), then the cp (echo
329     OK).
330     with patch.object(r, "_wsl_bash",
331                       side_effect=[_proc(0, stdout="READABLE"), _proc(0, stdout="OK")):
332         staged = r._stage_video(r"C:\v\clip.mp4", "clip.mp4")
333         assert staged == "~/clips/clip.mp4"
334
335 def test_stage_video_failure():
336     r = _runner()
337     # Source readable, but the cp itself fails -> None.
338     with patch.object(r, "_wsl_bash",
339                       side_effect=[_proc(0, stdout="READABLE"), _proc(1, stderr="disk
full"))]:
340         assert r._stage_video(r"C:\v\clip.mp4", "clip.mp4") is None
341
342
343 def test_stage_video_unreadable_source_fails_fast_with_actionable_msg(caplog):
344     """A source invisible to WSL (Google Drive / UNC) must fail BEFORE cp, with a clear,
345     fixable message ? not a cryptic 'cannot stat' that silently yields 0 rallies."""
346     r = _runner()
347     import logging
348     # Preflight `test -r` returns no READABLE -> unreadable. cp must NOT be attempted.
349     with patch.object(r, "_wsl_bash", return_value=_proc(1, stdout="")) as wsl, \
350         caplog.at_level(logging.ERROR):
351         assert r._stage_video("G:\My Drive\Sharing\clip.mp4", "clip.mp4") is None
352         assert wsl.call_count == 1 # only the preflight ran; cp short-circuited
353         assert "not readable from WSL" in caplog.text
354         assert "local disk" in caplog.text
355
356

```

```

357 def test_wsl_source_unreadable_reason_none_when_readable():
358     r = _runner()
359     with patch.object(r, "_wsl_bash", return_value=_proc(0, stdout="READABLE")):
360         assert r.wsl_source_unreadable_reason("/mnt/c/v/clip.mp4") is None
361
362
363 def test_wsl_source_unreadable_reason_none_on_flaky_probe():
364     """A timed-out/raising probe must NOT block a genuinely-readable source."""
365     r = _runner()
366     with patch.object(r, "_wsl_bash", side_effect=subprocess.TimeoutExpired(cmd="x",
367                                     timeout=1)):
368         assert r.wsl_source_unreadable_reason("/mnt/c/v/clip.mp4") is None
369
370 def test_run_predict_stages_when_enabled(tmp_path):
371     """With stage_in_wsl=True, run_predict should stage the video first."""
372     r = TrackNetRunner(TrackNetConfig(weights_path="/w/model.h5", stage_in_wsl=True))
373     out_dir = str(tmp_path)
374     csv_path = os.path.join(out_dir, "clip_predict.csv")
375     with patch.object(r, "_stage_video", return_value="~/clips/clip.mp4") as mock_stage,
376         \
377         patch.object(r, "_wsl_bash", return_value=_proc(0, stdout="done")):
378         open(csv_path, "w").close()
379         result = r.run_predict(r"C:\v\clip.mp4", out_dir)
380         mock_stage.assert_called_once()
381         assert result == csv_path
382
383 # ----- DetectorRunner ABC
384 contract
385
386 def test_tracknet_runner_implements_detector_runner_abc():
387     """TrackNetRunner must be a DetectorRunner (for polymorphism / future native
388     runners)."""
389     r = _runner()
390     assert isinstance(r, DetectorRunner)
391     # healthcheck contract: (bool, str) ? exercised with patch in other tests, but shape
392     here too
393     with patch.object(r, "_wsl_bash", return_value=_proc(0, stdout="TF ok")):
394         res = r.healthcheck()
395         assert isinstance(res, tuple) and len(res) == 2
396         assert isinstance(res[0], bool)
397         assert isinstance(res[1], str)
398
399 def test_wasb_runner_implements_detector_runner_abc():
400     """WasbRunner must also satisfy the DetectorRunner contract."""
401     w = WasbRunner(WasbConfig())
402     assert isinstance(w, DetectorRunner)
403     with patch.object(w, "_wsl_bash", return_value=_proc(0, stdout="torch ok")):
404         res = w.healthcheck()
405         assert isinstance(res, tuple) and len(res) == 2
406         assert isinstance(res[0], bool) and isinstance(res[1], str)
407
408 # ----- WASB streaming fast-path wiring
409
410 def test_wasb_config_stream_video_default_off_and_parsing():
411     """Streaming is OFF by default (production behavior unchanged) and opt-in via
412     cfg."""
413     assert WasbConfig().stream_video is False
414     cfg = WasbConfig.from_indexing_cfg({"wasb": {"stream_video": True}})
415     assert cfg.stream_video is True
416     assert WasbConfig.from_indexing_cfg({"wasb": {}}).stream_video is False

```

```

416
417 def _wasb_run_predict_cmd(stream_video, tmp_path):
418     """Drive WasbRunner.run_predict with WSL fully mocked and return the inference cmd
419     string (the last _wsl_bash call carries `python wasb_infer.py ...`)."""
420     r = WasbRunner(WasbConfig(stream_video=stream_video))
421     out_dir = str(tmp_path)
422     key = "clip__abc123abc123"
423     csv_path = os.path.join(out_dir, f"{key}_wasb.csv")
424     with patch.object(r, "_stage_infer_script", return_value=True), \
425         patch.object(r, "_stage_video", return_value="~/clips/clip.mp4"), \
426         patch.object(r, "_wsl_rm_rf"), \
427         patch.object(r, "_wsl_bash", return_value=proc(0, stdout="done")) as wsl:
428         open(csv_path, "w").close()
429         result = r.run_predict(r"C:\v\clip.mp4", out_dir, video_id="abc123abc123def")
430     assert result == csv_path
431     # The inference invocation is the call whose command contains wasb_infer.py.
432     infer_cmds = [c.args[0] for c in wsl.call_args_list if "wasb_infer.py" in c.args[0]]
433     assert len(infer_cmds) == 1
434     return infer_cmds[0]
435
436
437 def test_run_predict_streaming_passes_stream_flag_not_frames_dir(tmp_path):
438     cmd = _wasb_run_predict_cmd(stream_video=True, tmp_path=tmp_path)
439     assert "--stream-video" in cmd
440     assert "--frames_out_dir" not in cmd # no PNG extraction on the fast path
441     assert "--video " in cmd # still decodes the staged video
442
443
444 def test_run_predict_default_passes_frames_dir_not_stream_flag(tmp_path):
445     cmd = _wasb_run_predict_cmd(stream_video=False, tmp_path=tmp_path)
446     assert "--stream-video" not in cmd # default path unchanged
447     assert "--frames_out_dir" in cmd
448
449
450 # (Intentionally no "no-wsl unpatched predict" test here: the stage/inference paths
451 # raise FileNotFoundError on this Linux box when "wsl" is absent, while only the
452 # healthcheck paths catch it today. The graceful-None cases are already exercised
453 # by the patched tests above and by the hybrid tests. Adding a broad except in
454 # run_predict would be a separate robustness change.)
455

```

7. user (2026-06-19T16:40:16.983Z)

<persisted-output>

Output too large (22.7KB). Full output saved to: C:\Users\avidu\claude\projects\C--Users-avidu-Projects-badminton-highlight-indexer\8d4866bf-8edd-4da9-909e-022ddee12675\tool-results\toolu_01Gp8Sb5orZ4h2PPbuUriezbt.txt

Preview (first 2KB):

```

config.json:30:         "dead_time_merge_gap": 2.0,
README.md:175:         "dead_time_merge_gap": 2.0,
artifacts\long_rally_lens_n15.json:5:         "merge_gap": 2.0,
artifacts\long_rally_lens_n15.json:10:        "inactivity_rally_thresh": 0.2,
artifacts\long_rally_lens_n15.json:11:        "inactivity_min_density": 0.3,
artifacts\long_rally_lens_n15.txt:1:# Rally-segmentation eval ? windowing 'served' (vt=0.02,
merge_gap=2.0, min_win=2.0, cap=6.0, inact_end=1.5/rt=0.2/md=0.3), IoU=0.5
docs\ANALYZERS_AND_RECONCILERS.md:367:| 3 | `over_merged` | a dead-time gap **>`merge_gap` of
shuttle-inactivity sits **inside** one segment | **split** at the gap |
eval_baselines\nightly_baseline.json:180:        "inactivity_min_density": 0.34,
eval_baselines\nightly_baseline.json:181:        "inactivity_rally_thresh": 0.08,
eval_baselines\nightly_baseline.json:183:        "merge_gap": 2.0,
eval_baselines\nightly_baseline.json:365:        "inactivity_min_density": 0.3,
eval_baselines\nightly_baseline.json:366:        "inactivity_rally_thresh": 0.2,
eval_baselines\nightly_baseline.json:368:        "merge_gap": 2.0,
eval_baselines\nightly_baseline.json:550:        "inactivity_min_density": 0.3,

```

```

eval_baselines\nightly_baseline.json:551:         "inactivity_rally_thresh": 0.2,
eval_baselines\nightly_baseline.json:553:         "merge_gap": 2.0,
docs\CODE_MAP.md:87: -> TrackNetRunner.trajectory_to_action_windows(points, fps, frame_width,
chunk_start, velocity_thresh, merge_gap, min_window_duration, chunk_index) [the model-agnostic
WINDOWING BRAIN]
docs\CODE_MAP.md:129:[Omitted long matching line]
docs\CODE_MAP.md:181:[Omitted long matching line]
docs\CODE_MAP.md:237:`detection_params` keys by detector:
wasb={`{detector,velocity_thresh,merge_gap,min_action_window_duration,weights_path}`}; tracknet
adds `queue_length`; yolo={`{velocity_thresh,merge_gap,frame_skip,tracker,detector_model,detector
_score_threshold,min_action_window_duration}`}.
...
</persisted-output>

```

8. assistant (2026-06-19T16:40:20.355Z)

Now I have the core code and tests. Let me look at how the windowing is actually called in production (the hybrid segmenter), to verify the production config values for `merge\_gap`, `window\_inactivity\_end`, and whether R4 is even ON by default.

9. user (2026-06-19T16:40:22.821Z)

```

backend/config/models.py:149:     # window stays dense with FAST motion (velocity >
inactivity_rally_thresh). Only cuts.
backend/config/models.py:153:     window_inactivity_end: float = 1.5
backend/config/models.py:154:     inactivity_rally_thresh: float = 0.20
backend/config/models.py:155:     inactivity_min_density: float = 0.30
Binary file backend/config/__pycache__/models.cpython-313.pyc matches
Binary file backend/detectors/__pycache__/rally_gate.cpython-313.pyc matches
Binary file backend/detectors/__pycache__/tracknet_runner.cpython-313.pyc matches
Binary file backend/detectors/__pycache__/wasb_runner.cpython-313.pyc matches
backend/eval/calibrate_local.py:6:     WASB trajectory CSV ->
trajectory_to_action_windows(velocity, merge_gap, min_dur)
backend/eval/calibrate_local.py:46:         windows =
TrackNetRunner.trajectory_to_action_windows(
backend/eval/calibrate_wasb.py:5:     -> TrackNetRunner.trajectory_to_action_windows(cfg)
[predict_fn]
backend/eval/calibrate_wasb.py:30:BASELINE: Config = (0.02, 2.0, 2.0) # the WasbConfig /
trajectory_to_action_windows defaults
backend/eval/calibrate_wasb.py:50:         wins = TrackNetRunner.trajectory_to_action_windows(
backend/eval/distill_local.py:57:     wins = TrackNetRunner.trajectory_to_action_windows(
backend/eval/distill_local.py:79:     wins = TrackNetRunner.trajectory_to_action_windows(
backend/eval/golden_fixtures.py:33: takes only the 3-tuple ``(vt, mg, mwd)`` and
``TrackNetRunner.trajectory_to_action_windows``
backend/eval/rally_seg_eval.py:350:
bounded.append(f"inact_end={p['inactivity_end']}/rt={p['inactivity_rally_thresh']}")
backend/eval/rally_seg_eval.py:351:         f"/md={p['inactivity_min_density']}")
backend/eval/rally_seg_eval.py:445:     if args.inactivity_rally_thresh is not None:
backend/eval/rally_seg_eval.py:446:         preset = replace(preset,
inactivity_rally_thresh=args.inactivity_rally_thresh)
backend/eval/rally_seg_eval.py:447:     if args.inactivity_min_density is not None:
backend/eval/rally_seg_eval.py:448:         preset = replace(preset,
inactivity_min_density=args.inactivity_min_density)
backend/eval/served_gate_a.py:8: ``trajectory_to_action_windows`` at the production operating
point
backend/eval/served_gate_a.py:66:     ``trajectory_to_action_windows`` operating point. No GPU,
no Gemini: the person cue is OFF.
backend/eval/served_gate_a.py:69:         windows = TrackNetRunner.trajectory_to_action_windows(
backend/eval/windowing.py:54:     knobs ``TrackNetRunner.trajectory_to_action_windows`` takes, so
a preset *is* the windowing.
backend/eval/windowing.py:71:     inactivity_rally_thresh: float = 0.08
backend/eval/windowing.py:72:     inactivity_min_density: float = 0.34
backend/eval/windowing.py:82:     ``trajectory_to_action_windows`` / ``distill_local`` call
sites already speak. (The
backend/eval/windowing.py:95:         "inactivity_rally_thresh":

```

```

self.inactivity_rally_thresh,
backend/eval/windowing.py:96:         "inactivity_min_density": self.inactivity_min_density,
backend/eval/windowing.py:122:         inactivity_end=float(idx.window_inactivity_end),      # R4
rally-end trim (default 0=OFF)
backend/eval/windowing.py:123:
inactivity_rally_thresh=float(idx.inactivity_rally_thresh),
backend/eval/windowing.py:124:         inactivity_min_density=float(idx.inactivity_min_density),
backend/eval/windowing.py:164:         inactivity_end=float(idx.get("window_inactivity_end",
SERVED.inactivity_end)),
backend/eval/windowing.py:165:
inactivity_rally_thresh=float(idx.get("inactivity_rally_thresh",
SERVED.inactivity_rally_thresh)),
backend/eval/windowing.py:166:
inactivity_min_density=float(idx.get("inactivity_min_density", SERVED.inactivity_min_density)),
backend/eval/windowing.py:190:     A thin wrapper over
``TrackNetRunner.trajectory_to_action_windows`` that takes a named preset
backend/eval/windowing.py:196:     return TrackNetRunner.trajectory_to_action_windows(
backend/eval/windowing.py:201:         window_inactivity_end=preset.inactivity_end,

```

10. assistant (2026-06-19T16:40:25.147Z)

Important - there are two paths. Let me check the production config defaults
(`backend/config/models.py`) and how the production hybrid segmenter calls the windowing.

11. user (2026-06-19T16:40:26.582Z)

```

120         motion_threshold: float = 0.08
121         min_segment_duration: float = 3.0
122         dead_time_merge_gap: float = 2.0
123         chunk_duration: float = 90.0
124         chunk_overlap: float = 10.0
125         detector_model: str = "fasterrcnn_resnet50_fpn"
126         detector_score_threshold: float = 0.5
127         yolo_velocity_threshold: float = 0.15
128         yolo_frame_skip: int = 3
129         yolo_tracker: str = "bytetrack.yaml"
130         yolo_prefetch_workers: int = 2
131         min_action_window_duration: float = 2.0
132         # BOUNDED WINDOWING (over-merge fix W1, RALLY_QUALITY_RESEARCH.md §6). 0 = OFF; > 0
= a window
133         # longer than this many seconds is recursively split at its largest internal
inactivity gap (?
134         # ``window_min_split_gap`` s ? the most likely rally boundary), so one window can't
swallow
135         # several rallies + the dead time between them. Never merges, only cuts (recall
can't drop).
136         # *** R1 MILESTONE DEFAULT-ON (cap=6): the smallest safe local-windowing flip.
Measured n=13
137         # golden, R2 tolF1: baseline?milestone (cap=6 + R4 below) ?+0.222, sign 12/0,
p=0.0005; cap=6
138         # beats cap=12 ?+0.110, sign 12/0; net over-seg guard PASS (mean merge_rate
0.651?0.161). ***
139         max_window_duration: float = 6.0
140         window_min_split_gap: float = 0.5 # min internal gap (s) that qualifies as a split
point
141         # HARD-CAP fallback for W1: when True, an over-long window with NO internal
inactivity gap
142         # (a continuously-active trajectory ? e.g. the real-footage mahadevpura-1 174s blob)
is
143         # chopped at its midpoint until ? max_window_duration, making the cap a true upper
bound.
144         # Only cuts (recall can't drop). OFF by default until measured/promoted.
145         window_hard_cap: bool = False
146         # R4 rally-END inactivity trim (s, 0 = OFF). After a rally the shuttle keeps moving
slowly

```

```

147      # (picked up / walked) above velocity_thresh, so the window over-extends its END
(the measured
148      # [?end] gap vs golden). Trim the post-rally tail back to the last frame whose
trailing
149      # window stays dense with FAST motion (velocity > inactivity_rally_thresh). Only
cuts.
150      # *** R1 MILESTONE DEFAULT-ON (1.5/0.20/0.30): on top of cap=6, R4 adds a small but
significant
151      # end-precision gain ? ?tolF1 +0.040, sign 9/1/3, p=0.022, [?end] 4.96?3.31s (n=13).
The W1 cap
152      # is the dominant lever; R4 is the marginal increment. See QUALITY_ITERATIONS.md
"R1". ***
153      window_inactivity_end: float = 1.5
154      inactivity_rally_thresh: float = 0.20
155      inactivity_min_density: float = 0.30
156      # R5 blob-fallback (default OFF). LAST-RESORT splitter for the continuously-active
blob: after
157      # the W1 gap-split AND the R4 inactivity trim have each had their chance, a group
STILL longer
158      # than window_blob_factor × max_window_duration (no internal gap, no rally-end
signal ? e.g.
159      # mahadevpura-1's ~65s residual ? 11× a 6s cap) is midpoint-chopped to ? the cap as
a FORCED cut
160      # (logged + flagged on the window) so the degenerate single-window outcome is
avoided and the
161      # clip is surfaced as needing rally-STATE cues, not a bigger cap. Differs from
window_hard_cap
162      # (which chops BEFORE R4 and over-segments normal clips): blob-fallback runs AFTER
R4 and only
163      # force-cuts the genuine blob (the factor gate). Only cuts (recall can't drop).
164      window_blob_fallback: bool = False
165      # Magnitude gate that keeps R5 surgical: only a group longer than this ×
max_window_duration is a
166      # blob (a normal long rally ~1-2× the cap is left alone; the mahadevpura-1 residual
is ~11×).
167      # Default 4.0 is do-no-harm on the golden corpus (rescues only the continuously-
active
168      # clips, every other clip a no-op within noise) ? see docs/QUALITY_ITERATIONS.md
"R5".
169      window_blob_factor: float = 4.0
170      log_yolo_candidates: bool = True
171      store_yolo_candidates: bool = True
172      ai_handoff_padding: float = 2.0
173      ai_max_workers: int = 5
174      # Width (px) the gemini segmenter re-encodes chunks to before upload. Stream-copied
175      # chunks of a high-bitrate (4K) source blow past the provider upload cap
176      # (backend/providers/gemini.py::MAX_UPLOAD_MB) and are refused ? observed live as
177      # "0 segments / success". Re-encoding also makes chunk boundaries frame-accurate
178      # (stream copy cuts at the nearest keyframe). 0 = legacy stream-copy at source res.
179      # Matches gemini_refine's --clip-scale-width default.

```

12. user (2026-06-19T16:40:27.648Z)

```

backend/pipeline/segmenters/trajectory_hybrid.py:167:         window_inactivity_end =
idx_cfg.get("window_inactivity_end", 0.0)
backend/pipeline/segmenters/trajectory_hybrid.py:206:         all_candidate_windows =
runner.trajectory_to_action_windows(
backend/pipeline/segmenters/trajectory_hybrid.py:212:
window_inactivity_end=window_inactivity_end,
backend/pipeline/segmenters/yolo_hybrid.py:151:         windows =
self.person.extract_action_windows_from_chunk(
backend/pipeline/segmenters/yolo_hybrid.py:176:         windows =
self.person.extract_action_windows_from_chunk(
Binary file backend/pipeline/segmenters/__pycache__/trajectory_hybrid.cpython-313.pyc matches
Binary file backend/pipeline/segmenters/__pycache__/yolo_hybrid.cpython-313.pyc matches

```

13. assistant (2026-06-19T16:40:31.375Z)

R4 is DEFAULT-ON in production (`window\_inactivity\_end: float = 1.5`, `inactivity\_rally\_thresh: 0.20`, `inactivity\_min\_density: 0.30`). This is critical for the claim's significance. Let me read the production caller in `trajectory\_hybrid.py` to confirm the exact wiring, and verify how `merge\_gap` is passed there.

14. user (2026-06-19T16:40:32.298Z)

```
150
151     video_duration = get_video_duration(video_path)
152     if video_duration <= 0:
153         logger.error("Invalid video duration.")
154         return []
155     fps, frame_width = self._probe_fps_width(video_path)
156
157     # --- Runner config + windowing thresholds ---
158     runner, runner_params = self._build_runner(idx_cfg)
159
160     velocity_thresh = idx_cfg.get(self.family, {}).get("velocity_threshold", 0.02)
161     merge_gap = idx_cfg.get("dead_time_merge_gap", 2.0)
162     min_window_duration = idx_cfg.get("min_action_window_duration", 2.0)
163     # Bounded-windowing over-merge fix (W1): 0 = OFF (unchanged). See
IndexingConfig.
164     max_window_duration = idx_cfg.get("max_window_duration", 0.0)
165     window_min_split_gap = idx_cfg.get("window_min_split_gap", 0.5)
166     window_hard_cap = idx_cfg.get("window_hard_cap", False)
167     window_inactivity_end = idx_cfg.get("window_inactivity_end", 0.0)
168     inactivity_rally_thresh = idx_cfg.get("inactivity_rally_thresh", 0.08)
169     inactivity_min_density = idx_cfg.get("inactivity_min_density", 0.34)
170     window_blob_fallback = idx_cfg.get("window_blob_fallback", False)
171     window_blob_factor = idx_cfg.get("window_blob_factor", 4.0)
172     handoff_padding = idx_cfg.get("ai_handoff_padding", 2.0)
173     ai_max_workers = idx_cfg.get("ai_max_workers", 5)
174     log_candidates = idx_cfg.get("log_yolo_candidates", True)
175     store_candidates = idx_cfg.get("store_yolo_candidates", True)
176
177     detection_params = {
178         "detector": self.name,
179         "velocity_thresh": velocity_thresh,
180         "merge_gap": merge_gap,
181         "min_action_window_duration": min_window_duration,
182         **runner_params,
183     }
184
185     # --- Phase 1: env healthcheck (fail fast with a clear message) ---
186     ok, msg = runner.healthcheck()
187     if not ok:
188         logger.error("%s WSL environment not ready: %s", label, msg)
189         return []
190     logger.info("%s WSL env OK: %s", label, msg)
191
192     # --- Phase 2: trajectory -> candidate rally windows ---
193     # Trajectory detectors process the whole video in one pass (they carry
194     # their own frame buffering), so unlike yolo_hybrid we don't pre-chunk.
195     t_start = time.time()
196     out_dir = os.path.join("output", self.family)
197     csv_path = runner.run_predict(video_path, out_dir, video_id=video_id)
198     if not csv_path:
199         logger.error("%s produced no trajectory; aborting.", label)
200         return []
201
202     points = runner.parse_trajectory_csv(csv_path)
203     logger.info("Parsed %d trajectory points (%d visible).",
204                 len(points), sum(1 for p in points if p.visible))
```

```

205
206     all_candidate_windows = runner.trajectory_to_action_windows(
207         points, fps=fps, frame_width=frame_width, chunk_start=0.0,
208         velocity_thresh=velocity_thresh, merge_gap=merge_gap,
209         min_window_duration=min_window_duration, chunk_index=0,
210         max_window_duration=max_window_duration, min_split_gap=window_min_split_gap,
211         window_hard_cap=window_hard_cap,
212         window_inactivity_end=window_inactivity_end,
213         inactivity_rally_thresh=inactivity_rally_thresh,
214         inactivity_min_density=inactivity_min_density,
215         window_blob_fallback=window_blob_fallback,
216         window_blob_factor=window_blob_factor,
217     )
218     logger.info("Phase 2 (%s) completed in %.1fs. Candidate windows: %d",
219                 label, time.time() - t_start, len(all_candidate_windows))
220
221     if log_candidates:
222         for cw in all_candidate_windows:
223             logger.info(f"[{label} rally]
224             {_fmt_ts(cw['start'])}-{_fmt_ts(cw['end'])} "
225             f"({cw['end'] - cw['start']:.1f}s) |
frames={cw['active_frame_count']} "
226             f"peak_v={cw['peak_velocity']:.3f}
mean_v={cw['mean_velocity']:.3f}")
227
228     # Per-window stats via the enrichment hook ? base = the 4 motion keys; the
fusion
229     # segmenter adds net_crossings + person-cue features. Computed once, reused for
both
230     # persistence and the handoff gate below.

```

15. assistant (2026-06-19T16:40:36.173Z)

Note: in `trajectory\_hybrid.py` the local defaults in `idx\_cfg.get(...)` are the OLD defaults (0.0/OFF, 0.08, 0.34), but the typed `IndexingConfig` (models.py) has R4 ON (1.5/0.20/0.30). The actual production value depends on what's in the config. Let me verify what `idx\_cfg` actually carries in production - whether the typed config defaults flow through. Let me check `config.json` and how idx_cfg is built.

16. user (2026-06-19T16:40:37.840Z)

```

30:         "dead_time_merge_gap": 2.0,
=== config.json indexing block ===
        255
        ],
        "court_pixels_min_ratio": 0.05
    }
},
"indexing": {
    "default_segmenter": "gemini",
    "use_gpu": true,
    "motion_threshold": 0.08,
    "min_segment_duration": 3.0,
    "dead_time_merge_gap": 2.0,
    "chunk_duration": 90,
    "chunk_overlap": 10,
    "detector_model": "fasterrcnn_resnet50_fpn",
    "detector_score_threshold": 0.5,
    "yolo_velocity_threshold": 0.15,
    "yolo_frame_skip": 3,
    "yolo_tracker": "bytetrack.yaml",
    "yolo_prefetch_workers": 2,
    "min_action_window_duration": 2.0,
    "log_yolo_candidates": true,
    "store_yolo_candidates": true,

```



```

"ai_handoff_padding": 2.0,
"ai_max_workers": 1,
"tracknet": {
    "wsl_distro": "Ubuntu",
    "repo_dir": "~/models/TrackNetV4",
    "conda_sh": "~/miniconda3/etc/profile.d/conda.sh",
    "conda_env": "TrackNetV4",
    "weights_path": "",
    "queue_length": 5,
    "stage_in_wsl": true,
    "wsl_stage_dir": "~/clips",
    "timeout_sec": 1800,
    "velocity_threshold": 0.02
},
"wasb": {
    "wsl_distro": "Ubuntu",
    "repo_dir": "~/models/WASB-SBDT",
    "conda_sh": "~/miniconda3/etc/profile.d/conda.sh",
    "conda_env": "wasb",

```

17. assistant (2026-06-19T16:40:42.014Z)

`config.json` does NOT set `window\_inactivity\_end`, `max\_window\_duration`, etc. So the effective production value depends on whether `idx\_cfg` is the raw dict from `config.json` (in which case R4 is OFF via the `.get(..., 0.0)` fallback) or the typed `IndexingConfig` serialized to a dict (R4 ON at 1.5). Let me find how `idx\_cfg` is constructed and passed into the segmenter.

18. user (2026-06-19T16:40:43.455Z)

```

15:- ``_build_runner(idx_cfg)`` ? returns (DetectorRunner, detector-specific
53:   def _build_runner(self, idx_cfg: Dict[str, Any]) -> Tuple[DetectorRunner, Dict[str,
Any]]:
84:       idx_cfg: Dict[str, Any]) -> Dict[str, Any]:
98:       idx_cfg: Dict[str, Any]) -> List[int]:
121:   idx_cfg = self.config.get("indexing", {})
127:   skip_ai = bool(idx_cfg.get("skip_ai_handoff", False))
128:   provider_name = self.config.get("ai_provider", idx_cfg.get("ai_provider", "gemini"))
135:   model_name = self.config.get("ai_model", idx_cfg.get("ai_model",
"gemini-2.5-flash"))
158:   runner, runner_params = self._build_runner(idx_cfg)
160:   velocity_thresh = idx_cfg.get(self.family, {}).get("velocity_threshold", 0.02)
161:   merge_gap = idx_cfg.get("dead_time_merge_gap", 2.0)
162:   min_window_duration = idx_cfg.get("min_action_window_duration", 2.0)
164:   max_window_duration = idx_cfg.get("max_window_duration", 0.0)
165:   window_min_split_gap = idx_cfg.get("window_min_split_gap", 0.5)
166:   window_hard_cap = idx_cfg.get("window_hard_cap", False)
167:   window_inactivity_end = idx_cfg.get("window_inactivity_end", 0.0)
168:   inactivity_rally_thresh = idx_cfg.get("inactivity_rally_thresh", 0.08)
169:   inactivity_min_density = idx_cfg.get("inactivity_min_density", 0.34)
170:   window_blob_fallback = idx_cfg.get("window_blob_fallback", False)
171:   window_blob_factor = idx_cfg.get("window_blob_factor", 4.0)

```

19. assistant (2026-06-19T16:40:48.035Z)

So `idx\_cfg = self.config.get("indexing", {})` ? the raw config dict. Let me check how `self.config` is built ? whether it's the raw JSON or the typed `IndexingConfig` merged in. This determines whether R4 is actually ON in production.

20. user (2026-06-19T16:40:49.396Z)

```

110     # but every live segmenter still returns a bare list ? the documented
111     # deliberate-incremental gap (CODE_MAP gotchas; main.py handles both).
112     label = self.display_label
113     # --- Sport profile + AI provider wiring (identical across segmenters) ---

```

```

114         sport_name = self.config.get("sport", "badminton")
115         try:
116             sport_profile = sport_registry.get(sport_name)
117         except KeyError as e:
118             logger.error(str(e))
119         return []
120
121         idx_cfg = self.config.get("indexing", {})
122         # Fully-local, NO-AI mode (default OFF): the trajectory candidate windows become
the
123         # rallies directly ? Phase 3's AI handoff is skipped, so no provider / API key
is
124         # needed and nothing is uploaded. Used to run the local detector standalone
(e.g. to

```

21. user (2026-06-19T16:40:51.165Z)

```

backend/config/loader.py:50:     return AppConfig().model_dump(mode="json")
backend/config/loader.py:109:    cfg_path.write_text(cfg.model_dump_json(indent=4),
encoding="utf-8")
backend/config/models.py:310:    # via model_dump, so legacy chained access keeps working ?
returning the Pydantic
backend/config/models.py:330:    return val.model_dump(mode="json")
backend/eval/served_gate_a.py:36:# TrajectoryHybridSegmenter.process_video reads for the served
fusion run).
backend/eval/windowing.py:7:(``TrajectoryHybridSegmenter.process_video``) windows COARSELY
(``vt=0.02, merge_gap=2.0,
backend/eval/windowing.py:8:min_window=2.0`` ? the ``config.indexing`` defaults). A cue measured
on the proposal set is
backend/eval/windowing.py:105:    ``TrajectoryHybridSegmenter.process_video`` reads exactly
these three:
backend/eval/windowing.py:149:    Reproduces ``TrajectoryHybridSegmenter.process_video``'s knob
reads exactly, against an
backend/pipeline/segmenters/fusion_hybrid.py:5:do ? it subclasses the same
`TrajectoryHybridSegmenter` engine. It then ENRICHES each
backend/pipeline/segmenters/fusion_hybrid.py:31:from
backend.pipeline.segmenters.trajectory_hybrid import TrajectoryHybridSegmenter
backend/pipeline/segmenters/fusion_hybrid.py:39:class
FusionSegmenter(TrajectoryHybridSegmenter):
backend/pipeline/segmenters/tracknet_hybrid.py:10:lives in
trajectory_hybrid.TrajectoryHybridSegmenter; this class only wires the
backend/pipeline/segmenters/tracknet_hybrid.py:16:from
backend.pipeline.segmenters.trajectory_hybrid import TrajectoryHybridSegmenter
backend/pipeline/segmenters/tracknet_hybrid.py:21:class
TrackNetHybridSegmenter(TrajectoryHybridSegmenter):
backend/pipeline/segmenters/trajectory_hybrid.py:44:class
TrajectoryHybridSegmenter(BasePlaySegmenter):
backend/pipeline/segmenters/wasb_hybrid.py:12:lives in
trajectory_hybrid.TrajectoryHybridSegmenter; this class only wires the
backend/pipeline/segmenters/wasb_hybrid.py:18:from backend.pipeline.segmenters.trajectory_hybrid
import TrajectoryHybridSegmenter
backend/pipeline/segmenters/wasb_hybrid.py:23:class
WasbHybridSegmenter(TrajectoryHybridSegmenter):
backend/reporting/__init__.py:45:    if hasattr(config, "model_dump"):
backend/reporting/__init__.py:47:        config = config.model_dump(mode="json")

```

22. assistant (2026-06-19T16:40:54.897Z)

Let me check `config/loader.py` line 50 ? `AppConfig().model\_dump(mode="json")` is the default config. This means the loaded config dict WILL carry the typed defaults (window_inactivity_end=1.5 etc.) from `IndexingConfig`. Let me confirm the loader merges the file over the typed defaults.

23. user (2026-06-19T16:40:55.387Z)

```

1      """Configuration loader and saver.
2
3      Provides a single place to load validated AppConfig with built-in defaults
4      merged with optional file overrides (config.json).
5
6      This replaces scattered json.load + .get() patterns throughout the codebase.
7      """
8
9      from __future__ import annotations
10
11     import json
12     import logging
13     from pathlib import Path
14     from typing import Any
15
16     from pydantic import BaseModel
17
18     from .models import AppConfig
19
20     logger = logging.getLogger(__name__)
21
22     DEFAULT_CONFIG_PATH = Path("config.json")
23
24
25     def _unknown_key_paths(data: dict[str, Any], model_cls: type[BaseModel], prefix: str =
26     """) -> list[str]:
27         """Dotted paths in `data` that the typed config will not honor.
28
29         Two silent failure modes exist without this check: a typo'd TOP-LEVEL key is
30         kept by AppConfig's ``extra="allow"`` but never read by typed code, and a
31         typo'd key INSIDE a section is silently dropped by Pydantic. Either way the
32         user's intent is lost ? collect every such key so load_config can warn.
33         """
34         unknown: list[str] = []
35         fields = model_cls.model_fields
36         for key, val in data.items():
37             if key not in fields:
38                 unknown.append(prefix + key)
39                 continue
40             ann = fields[key].annotation
41             if isinstance(val, dict) and isinstance(ann, type) and issubclass(ann,
42             BaseModel):
43                 unknown.extend(_unknown_key_paths(val, ann, f"{prefix}{key}.")
44         return unknown
45
46
47     def _get_builtin_defaults() -> dict[str, Any]:
48         """Return a plain dict of the current built-in defaults.
49
50         The AppConfig model is the single source of truth for defaults.
51         """
52         return AppConfig().model_dump(mode="json")
53
54
55     def load_config(path: str | Path | None = None) -> AppConfig:
56         """Load configuration with the following precedence:
57
58         1. Built-in defaults defined in the Pydantic models.
59         2. Values from the JSON file (if present and readable).
60         3. (Future) environment / CLI overrides.
61
62         Missing file or unreadable file ? returns a fully defaulted AppConfig.
63         Unknown keys in the file are tolerated (extra="allow" on the model).
64         """
65         cfg_path = Path(path) if path is not None else DEFAULT_CONFIG_PATH

```

```

64
65     file_data: dict[str, Any] = {}
66     if cfg_path.exists():
67         try:
68             with open(cfg_path, "r", encoding="utf-8") as f:
69                 file_data = json.load(f)
70         except Exception as exc:
71             # Non-fatal: continue with defaults + whatever we could parse.
72             # In production we might log here.
73             logger.warning(f"Failed to read {cfg_path}: {exc}. Using defaults + partial
data.")
74
75     # A config.json that is valid JSON but not an OBJECT (e.g. `[ ]`, `42`, `"x"`,
`null`) would
76     # otherwise crash startup: a truthy non-mapping hits `.items()` in
`_unknown_key_paths`, and any
77     # non-mapping breaks the `{**defaults, **file_data}` unpack. Degrade to defaults +
warn, per the
78     # loader's "bad input is non-fatal" contract.
79     if not isinstance(file_data, dict):
80         logger.warning("%s is not a JSON object (got %s); ignoring it and using
defaults.",
81                        cfg_path, type(file_data).__name__)
82         file_data = {}
83
84     if file_data:
85         unknown = _unknown_key_paths(file_data, AppConfig)
86         if unknown:
87             logger.warning(
88                 "[CONFIG WARNING] %s contains keys the typed config does not "
89                 "recognize (typo'd or removed knobs ? top-level extras are kept "
90                 "but unread; unknown section keys are silently dropped): %s",
91                 cfg_path, ", ".join(sorted(unknown)),
92             )
93
94     # Merge: start from defaults, overlay file data (Pydantic will validate/coerce).
95     merged = {**_get_builtin_defaults(), **file_data}
96
97     return AppConfig.model_validate(merged)
98
99
100 def save_default_config(path: str | Path | None = None) -> Path:
101     """Write a fresh default config.json.
102
103     Used by scripts/doctor.py diagnostics and the --setup CLI path.
104     Overwrites if the file exists.
105     """
106     cfg_path = Path(path) if path is not None else DEFAULT_CONFIG_PATH
107     cfg = AppConfig()
108     cfg_path.parent.mkdir(parents=True, exist_ok=True)
109     cfg_path.write_text(cfg.model_dump_json(indent=4), encoding="utf-8")
110     return cfg_path
111
112
113 # Backwards-compatible alias used by some older call sites during transition.
114 def get_default_config() -> dict[str, Any]:
115     """Return defaults as a plain dict (for transitional use only)."""
116     return _get_builtin_defaults()
117

```

24. assistant (2026-06-19T16:41:01.620Z)

One subtlety: `merged = {\*\*\_get\_builtin\_defaults(), \*\*file\_data}` is a TOP-LEVEL merge. `file\_data["indexing"]` (from config.json) would REPLACE the entire defaulted `indexing` section, not deep-merge. But Pydantic's `model\_validate(merged)` re-applies field defaults for

keys absent from the supplied `indexing` dict. So `window\_inactivity\_end` absent from config.json's `indexing` block ? Pydantic fills it from `IndexingConfig` default = 1.5. Then the segmenter reads `self.config` which is the validated config. Let me confirm how `self.config` reaches the segmenter (is it the model_dump of the validated AppConfig, or the raw file dict).

25. user (2026-06-19T16:41:03.414Z)

```
backend/main.py:27:from backend.pipeline.segmenters.base import segmenter_registry
backend/main.py:31:from backend.config import load_config as load_app_config, AppConfig,
StitchingConfig # new typed config
backend/main.py:83:# New code should import load_app_config / AppConfig from backend.config
directly.
backend/main.py:84:def load_config(config_path: str = "config.json") -> Dict[str, Any]:
backend/main.py:172:     segmenter_cls = segmenter_registry.get(seg_name)
backend/main.py:176:     available = list(segmenter_registry.list_available().keys())
backend/main.py:194:     result = segmenter.process_video(video_id, req.video_path,
req.player_pool, req.max_frames)
backend/main.py:256:     if req.segmenter and not segmenter_registry.get(req.segmenter):
backend/main.py:257:         available = list(segmenter_registry.list_available().keys())
backend/main.py:405:     cfg = load_config() # legacy wrapper still works, returns dict for now
backend/main.py:480:     available_segmenters = list(segmenter_registry.list_available().keys())
backend/main.py:596:     segmenter_cls = segmenter_registry.get(seg_name)
backend/main.py:605:     result = segmenter.process_video(video_id, args.video_path,
max_frames=args.max_frames)
```

26. user (2026-06-19T16:41:07.361Z)

```
83     # New code should import load_app_config / AppConfig from backend.config directly.
84     def load_config(config_path: str = "config.json") -> Dict[str, Any]:
85         cfg = load_app_config(config_path)
86         return cfg.model_dump(mode="json")
87
88     # --- API Models ---
89     class ProcessRequest(BaseModel):
90         video_path: str
91         player_pool: Optional[List[str]] = None
92         max_frames: Optional[int] = None
93         segmenter: Optional[str] = None
94
95     class QueryRequest(BaseModel):
96         video_id: str
97         server: Optional[str] = None
98         receiver: Optional[str] = None
99         duration_min: Optional[float] = None
100         event_type: Optional[str] = None
101
102     class CompileRequest(BaseModel):
103         video_id: str
104         segment_ids: List[int]
105         output_filename: str
106
107     def _finalize_reingest(video_id: str, segments, play_wm: int, cand_wm: int) -> None:
108         """Commit a (re)segmentation with destroy-AFTER-confirm semantics.
109
110         On a non-empty run, drop the PRIOR rows (id <= the pre-run watermark) so only the
111         fresh segments remain. On an empty run, drop the NEW partial rows (id > watermark)
112         and
113         keep the prior good results intact ? a failed/empty re-run never destroys prior
114         data.
```

27. user (2026-06-19T16:41:07.937Z)

```
165         }
166         return response
```

```

167
168         # 2. Run segmenter & indexer
169         logger.info("[API] Video passed checks. Segmenting and indexing...")
170         db_instance.add_video(video_id, req.video_path, "processed", [r.model_dump() for
r in val_results])
171
172         segmenter_cls = segmenter_registry.get(seg_name)
173         if not segmenter_cls:
174             # Submit-time validation already 400s unknown names; this is the defensive
175             # in-worker path (e.g. config default pointing at an unregistered
segmenter).
176             available = list(segmenter_registry.list_available().keys())
177             response = {
178                 "video_id": video_id,
179                 "status": "failed",
180                 "message": f"Segmenter '{seg_name}' not found. Available segmenters:
{available}",
181                 "results": [r.model_dump() for r in val_results],
182                 "play_segments": [],
183             }
184             return response
185
186         logger.info(f"[API] Using segmenter: {seg_name}")
187         notify(f"segmenting ({seg_name})")
188         segmenter_actual = seg_name
189         # Destroy-AFTER-confirm: remember the pre-run rows, then keep new + drop old
only
190         # if the run yields segments; otherwise drop the new partial rows and keep the
old.
191         play_wm, cand_wm = db_instance.segment_watermarks(video_id)
192         segmenter = segmenter_cls(db_instance, global_config)
193         try:
194             result = segmenter.process_video(video_id, req.video_path, req.player_pool,
req.max_frames)
195         except Exception:
196             # A raising segmenter must not leave half-written rows: restore the pre-run
197             # state (same destroy-after-confirm contract as the Failure branch), then
let
198             # the job worker record the failure.
199             db_instance.prune_segments(video_id, play_after=play_wm, cand_after=cand_wm)

```

28. assistant (2026-06-19T16:41:11.073Z)

Let me confirm `global\_config` is the model_dump (typed defaults applied) and not the raw file dict.

29. user (2026-06-19T16:41:11.918Z)

```

62:global_config: Optional[AppConfig] = None # now typed (Pydantic model)
123:     return global_config
146:     val_results, val_context = registry.run_all_with_context(req.video_path, global_config)
150:     default_seg = getattr(getattr(global_config, "indexing", None), "default_segmenter",
"motion")
192:         segmenter = segmenter_cls(db_instance, global_config)
231:         db=db_instance, video_id=video_id, video_path=req.video_path,
config=global_config,
249:         allowed_root = getattr(getattr(global_config, "security", None), "allowed_video_root",
None)
298:# backend.main.global_config, so behavior ? route paths, status codes, body-size limits,
318:     output_dir = getattr(getattr(global_config, "output", None), "output_dir", "output") or
"output"
366:     stitching = getattr(global_config, "stitching", None) or StitchingConfig()
===
543:     global_config = load_app_config() # returns typed AppConfig

```

30. assistant (2026-06-19T16:41:15.863Z)

`global\_config` is the typed `AppConfig`. But the segmenter does `self.config.get("indexing", {})` ? that requires `self.config` to be dict-like. `AppConfig` is a Pydantic model. Let me check `BasePlaySegmenter.\_\_init\_\_` to see if it converts to a dict (model_dump) ? that determines whether R4 defaults flow through.

31. user (2026-06-19T16:41:17.633Z)

```
6:class BasePlaySegmenter(ABC):
7:    def __init__(self, db: Database, config: Dict[str, Any]):
9:        self.config = config
36:    def __init__(self):
=== models.py __getitem__/get ===

# Allow extra keys during transition so existing config.json files do not break.
model_config = {
    "extra": "allow",
    "validate_assignment": True,
}

# Temporary migration shim for legacy dict-style access (config.get("section",
{}).get(...)).
# Will be removed once all call sites (validators, segmenters, etc.) move to typed
# attribute access / the typed config. A nested *section* is returned as a plain dict
# via model_dump, so legacy chained access keeps working ? returning the Pydantic
# section model instead breaks it (a model has no .get), which bit every validator on
# real runs. Scalar leaves are returned as-is.
def get(self, key: str, default: Any = None) -> Any:
    _MISSING = object()
    val: Any = _MISSING
    if key in type(self).model_fields or hasattr(self, key):
        val = getattr(self, key, _MISSING)
    if val is _MISSING:
        # Legacy convenience: find a leaf key inside any nested section. Sections are
        # discovered dynamically from the model fields, so a future top-level section
        # (e.g. `audio`, `jobs`) is covered automatically without editing a hardcoded list.
        for fname in type(self).model_fields:
            section = getattr(self, fname, None)
            if isinstance(section, BaseModel) and hasattr(section, key):
                val = getattr(section, key)
                break
    if val is _MISSING:
        return default
    if isinstance(val, BaseModel):
        return val.model_dump(mode="json")
    return val

def __getitem__(self, key: str) -> Any:
    val = self.get(key)
    if val is None and key not in type(self).model_fields:
        # Re-raise like a real dict for unknown top-level
        raise KeyError(key)
    return val
```

32. assistant (2026-06-19T16:41:27.636Z)

Confirmed. `AppConfig.get("indexing", {})` returns the indexing section via `model\_dump(mode="json")`, which includes `window\_inactivity\_end=1.5`, `inactivity\_rally\_thresh=0.20`, `inactivity\_min\_density=0.30`, `max\_window\_duration=6.0`, `dead\_time\_merge\_gap=2.0`. So ****R4 is ON in production**** at the operating point (1.5s trail, density 0.30, rally_thresh 0.20).

Now let me critically test the claim's core scenario by actually running the loop logic. The key question: with `merge\_gap=2.0` (60 frames at 30fps), does a 1.8s visibility gap split the group

BEFORE the trail window ever sees an isolated frame? Note: `\_active\_frames` breaks the trajectory on invisibility (`prev=None`), but grouping uses `merge\_gap` on the *active frame indices*. Let me trace this precisely and also test the specific "win_count ~1-2 ? 100% density" sub-window scenario inside a single group.

33. user (2026-06-19T16:41:34.449Z)

group last frame = 114 -> trimmed last frame = 114
Did the isolated tail frame survive (inflate END)? True

34. assistant (2026-06-19T16:41:40.772Z)

The claim's mechanism is REAL when reproduced at the R4 boundary directly: an isolated fast frame after a sub-merge_gap dead window survives the trim because at frame 114 the trailing 45-frame window contains only that 1 frame ? density $1/1 = 1.0$? 0.30 ? marked dense ? END not trimmed. Now the critical question for production reachability: does `merge\_gap` grouping pre-empt this? The gap (54 frames) is < merge_gap (60 frames), so it does NOT split. But is the active-frame requirement different? `\_active\_frames` needs a *visible* frame with velocity computed from the *previous visible* frame. Let me test through the full public API with realistic invisibility, which is how production actually feeds it.

35. user (2026-06-19T16:41:47.331Z)

start=0.03 end=3.87 dur=3.83 frames=61
NOTE: real rally ends ~frame60 (2.0s); tail frames are at ~3.85s

36. assistant (2026-06-19T16:41:53.597Z)

Critical finding. Through the FULL public API at the production operating point, the window END is inflated to 3.87s when the real rally ended at ~2.0s ? a ~1.85s over-extension caused exactly by the mechanism the claim describes. Note the active-frame count is 61 (the tail pickup at 115/116 only contributes when velocity from the prior visible frame is computable; here frame 115's velocity comes from frame 60 across the gap). Let me verify the precise mechanism ? was the END set by the isolated tail surviving R4, and would the cap have split it. Let me also test whether the gap being *larger* than merge_gap pre-empts it (the claim's "somewhat mitigated by merge_gap").

37. user (2026-06-19T16:41:59.353Z)

```
<merge_gap gap=54fr(1.8s): 1 window(s): ['0.03-3.87']  
>merge_gap gap=72fr(2.4s): 0 window(s): []
```

38. assistant (2026-06-19T16:42:06.688Z)

This precisely confirms the claim's "somewhat mitigated by merge_gap" caveat AND shows it is INCOMPLETE mitigation:

- gap **< merge_gap (1.8s < 2.0s)**: same group ? END inflated to 3.87s (the bug fires).
- gap **> merge_gap (2.4s)**: grouping splits ? tail in its own group ? that tail group is too short (min_window_duration=2.0) ? dropped entirely ? 0 windows.

So the claim is correct that merge_gap grouping pre-empts the LARGE-gap case, but there is a real reachable band: **any dead-time gap shorter than `merge\_gap` (2.0s)** lets an isolated post-gap fast frame survive R4 and inflate the END. The claim's "1.8s" example sits exactly in this reachable band. Let me verify the cap (6.0s) doesn't rescue it (window is 3.83s < 6.0, so the cap doesn't fire) and check whether a *blob* of slow drift (the case R4 was designed for) is correctly trimmed ? to contrast normal R4 behavior vs the edge case.

39. user (2026-06-19T16:42:12.563Z)

dense-drift case: last frame 150 -> trimmed to 91 (correctly trims drift)
sparse-tail case: last frame 114 -> trimmed to 114 (does NOT trim - the quirk)

40. assistant (2026-06-19T16:42:19.967Z)

The contrast is clear and confirms the claim's mechanism precisely. The R4 trimmer's density denominator ``win_count = i - lo + 1`` counts only the *active frames inside the trailing window*, NOT the elapsed temporal frames. When the tail is sparse (a single fast frame after a sub-merge_gap dead window), `density = 1/1 = 1.0` and the END is NOT trimmed. When the tail is dense slow-drift (what R4 was designed to catch), density is correctly low and it IS trimmed.

Now let me assess the proposed FIX's boundary-shift risk. The proposal: change the denominator from "visible frame count" to "temporal length (total expected frames in trailing window)". Let me simulate what that does to a NORMAL clip where there's natural shuttle invisibility mid-rally (very common ? shuttle is small, gets lost frequently even during live play).

41. user (2026-06-19T16:42:30.237Z)

```
normal sparse-but-fast rally, real END frame=180
CURRENT (win_count denom): END frame = 180
PROPOSED (temporal denom): END frame = 180
=> proposed SHIFTS the END inward by 0 frames = 0.0 s
```

42. assistant (2026-06-19T16:42:34.483Z)

Interesting ? at 10 active-fps (1 every 3 frames), density over temporal = $15/45 = 0.33 \approx 0.30$, so it barely survives. Let me push to a more realistic sparser rate (shuttle is genuinely hard to track ? often <10 active fps) to show the boundary shift on a normal rally.

43. user (2026-06-19T16:42:40.748Z)

```
7.5 active-fps: real END=180  CURRENT END=180  PROPOSED END=12  shift=5.60s
6 active-fps:  real END=180  CURRENT END=180  PROPOSED END=5   shift=5.83s
5 active-fps:  real END=180  CURRENT END=180  PROPOSED END=0   shift=6.00s
```

44. assistant (2026-06-19T16:42:49.066Z)

This is the decisive finding. Switching the denominator to temporal length is NOT a safe edge-case hardening ? it is a ****silent, catastrophic boundary-behavior change****. On a perfectly normal, genuinely-fast rally where the tracker only emits a fast frame at ~5-7.5 active-fps (entirely realistic for shuttle detection ? the shuttle is small, motion-blurred, and frequently lost even during live play), the proposed temporal-denominator trim collapses the END from frame 180 (6s, correct) to frame 0-12 ? destroying the entire rally. The temporal density ``min_density=0.30`` was tuned against the *visible-frame* denominator; reusing it against a temporal denominator that's ~4-6x larger guarantees mass over-trimming.

This validates the owner's mandate exactly: any change to this denominator shifts rally-END boundaries on normal clips, requires re-tuning ``min_density``, and demands a measured Launch Report + dual-set regression ? NOT a blind code change. Let me confirm the test suite locks in the current ``win_count`` behavior and run the existing tests to confirm green baseline.

45. user (2026-06-19T16:42:51.400Z)

```
..... [100%]
40 passed in 0.16s
```

46. assistant (2026-06-19T16:42:57.508Z)

All 40 tests pass. The tests ``test_trim_inactive_tail_drops_slow_drift``, ``test_trim_inactive_tail_keeps_continuous_rally_and_is_noop_when_off``, and ``test_window_inactivity_end_trims_through_api`` lock in the current ``win_count``-denominator semantics. None of them cover the sparse-tail edge case, so the quirk is genuinely undocumented/untested but the existing density behavior is locked.

Let me do a final check on the "open question" in the plan ? whether the proposal addresses the ``min_density`` re-tuning that my simulation proves is mandatory. Let me check the QUALITY_ITERATIONS doc for how the current values were measured (to confirm the Launch-Report mandate applies here).

47. user (2026-06-19T16:42:58.762Z)

```
62:**Measured (n=15 golden, milestone cap6+R4, LOCAL vs golden; reproduce: `python -m
backend.eval.rally_seg_eval --stratify`):**
68:| all | all | 15 | 0.353 | 0.302 | 0.445 | 0.468 | 1.478 |
104:### R1 ? MILESTONE LAUNCHED (cap=6 + R4 flipped default-ON) ? re-verified at n=13
*(2026-06-18)*
105:The R1 milestone is **shipped** (PR #204): the W1 cap=6 + R4 inactivity-end defaults are now
ON.
116:| cap6 ? +R4 tolF1 | ?+0.047, 8/0, p=0.008 | ?+0.040, **9/1/3**, p=0.0215 |
123:W1 cap is the dominant lever / R4 is the marginal +0.040 increment (1 loss = GX010137, a
tolerance
129:The one clip the cap6+R4 milestone still fails is the **continuously-active blob**
(mahadevpura_1:
130:~174 s of play with no inactivity gap to split on). The W1 gap-split finds no gap; R4 finds
no slow
134:**R5** (`window_blob_fallback`, default-OFF) is the last resort: **after** the gap-split and
the R4
141:**Why a factor gate, and why not just `window_hard_cap`:** `hard_cap` chops **before** R4
and fires on
143:R4, and the magnitude gate (**default 4**) leaves a normal long rally (~1?2x the cap)
alone, force-cutting
146:**MEASURED (n=11, on the cap6+R4 milestone, R2 `tolF1`):**
163:### R1 ? milestone evidence (cap=6 + R4) + the net over-seg promotion guard *(2026-06-17)*
164:The R1 milestone is the smallest safe local-windowing flip: **W1 bounded windowing cap=6 +
the R4
165:rally-END inactivity trim, no W2** (the per-clip-best config, settled by R3/R4). This entry
(a) re-runs
176:| **+ R4 inact_end=1.5 (milestone)** | **0.323 [0.210, 0.433]** | **0.440** | **3.8 s** |
179:- **baseline ? milestone:** ?`tolF1` **+0.232** [+0.153, +0.304], **sign 10/0/1, p=0.002**
(the 1 tie is the
183:- **cap=6 ? +R4:** ?`tolF1` **+0.047** [+0.022, +0.072], **sign 8/0/3, p=0.008** (reproduces
the R3/R4 entry).
205:**R1 verdict:** the milestone (cap=6 + R4) clears the refined (net-guard) promotion bar ? a
significant `tolF1`
208:`inactivity_min_density: 0.30`) is **prepped on its own branch and left for the owner's
product-go** ? it changes
```

48. assistant (2026-06-19T16:43:05.132Z)

This is definitive. The R4 default-ON flip (PR #204) and its `inactivity\_min\_density: 0.30` were measured against the golden set (n=13/15) with a documented Launch Report (the dominant lever is the cap, R4 is the marginal +0.040 increment, sign 9/1/3, p=0.0215). The `min\_density=0.30` threshold is **calibrated against the current win_count denominator**. Changing the denominator to temporal length silently invalidates that calibration ? my simulation shows it would over-trim normal sparse-tracking rallies by up to 6 seconds (destroying entire rallies). This is squarely the kind of change the owner mandates must go through a measured Launch Report + dual-set regression, not a blind "harden the edge case" patch.

I have everything I need. Let me record the verdict.

49. user (2026-06-19T16:43:49.122Z)

Output does not match required schema: root: must NOT have additional properties

50. assistant (2026-06-19T16:43:51.853Z)

The schema doesn't accept `proposed\_fix`. Let me remove it.

51. user (2026-06-19T16:44:26.619Z)

Structured output provided successfully

52. assistant (2026-06-19T16:44:38.230Z)

I've completed the adversarial review of PR Claim B and returned the structured verdict. Summary of what I verified against the real code:

****Verdict: accurate**** ? the code description is exactly right. ``_trim_inactive_tail`` (backend/pipeline/detectors/tracknet_runner.py:402-428) computes density as ``fast_sum / win_count`` where ``win_count = i - lo + 1`` is the count of active frames in the trailing window, not elapsed time.

****Real but low-severity bug.**** I reproduced the inflation both directly and through the full public API at the production operating point (R4 is DEFAULT-ON: ``window_inactivity_end=1.5``, ``inactivity_min_density=0.30``, ``inactivity_rally_thresh=0.20``, confirmed flowing from ``IndexingConfig`` defaults through ``load_config`/`model_dump`` into ``trajectory_hybrid.py``). A rally + a 1.8s sub-``merge_gap`` invisibility gap + a sparse fast tail produced an END inflated to 3.87s vs a true ~2.0s. The claim's "mitigated by merge_gap" caveat is correct but partial: gaps ****larger**** than `merge_gap` (2.0s) split the group and drop the tail; gaps ****shorter**** than `merge_gap` leave the quirk fully reachable ? exactly where the 1.8s example sits.

****The proposed fix is the real concern.**** I simulated the temporal-denominator change keeping ``min_density=0.30``: it fixes the edge case but collapses normal genuinely-fast rallies (END frame 180 ? 0-12, a 5.6-6.0s regression) at realistic 5-7.5 active-fps shuttle tracking. ``min_density=0.30`` is calibrated against the visible-frame denominator and was flipped default-ON via a documented Launch Report (PR #204). So this is a silent rally-END boundary-behavior change masquerading as an edge-case hardening ? it must go through a measured Launch Report + dual-set golden regression with ``min_density`` re-tuned, not a blind code edit. The plan's own open question ("fall back for very few frames, or enforce minimum temporal density?") is unresolved, confirming the authors sensed but did not address this.

All 40 tests in `tests/test_tracknet_runner.py` pass; none cover the sparse-tail case, so a naive fix could pass CI while regressing real clips.